ABSCHNITT 01 · DAS FUNDAMENT

Webentwicklung

verstehen & selbst lernen

HTML · CSS · JAVASCRIPT

Ausführliche Erklärungen für verschiedene Webseiten

Handschriftliche Übungen mit dem S Pen

Zwischenprüfungen, Lösungen und Abschlusstest

Begleitende Praxisaufgaben für deine eigene Webseite

Dein persönliches Lern- und Arbeitsbuch

Passend zum Entwicklungsplan · Schritt 1 bis 5 von 25

AUSGABE 1.1 · OKTOBER 2026

So benutzt du dieses Arbeitsbuch

Dieses Buch behandelt Abschnitt 1 des Entwicklungsplans: Entwicklungswerkstatt (Schritt 1), HTML (Schritt 2), CSS (Schritt 3), JavaScript (Schritt 4) und ein erstes eigenständig nutzbares Mini-Projekt (Schritt 5). Das HTML-Wissen ist bewusst allgemein erklärt, damit du später auch völlig andere Webseiten bauen kannst.

Jede Lektion besteht aus einem Erklärungsteil und einer handschriftlich bearbeitbaren Übungsseite. Dort findest du eine unabhängige Denk- oder Codeaufgabe, einen Praxisauftrag für dein bestehendes Projekt sowie eine kurze Verständnisfrage. Die Antworten auf reine Wissensfragen stehen gesammelt ganz hinten.

DEIN LERNRHYTHMUS

Erst die Erklärung lesen, anschließend mit dem S Pen handschriftlich arbeiten, dann selbst in Codespaces programmieren und die Webseite im Browser testen. Danach das Ergebnis dokumentieren und anhand der Lernziele selbst überprüfen. Du musst Aufgaben nicht an einem Tag erledigen.

Das solltest du bereithalten

- Galaxy Tab S8 mit Tastatur und S Pen (alternativ PC oder anderes Tablet).
- Eine PDF-App, in der du auf Seiten schreiben kannst (zum Beispiel Samsung Notes nach dem Import einer PDF).
- Deinen Browser-Codeeditor mit einem Git-Repository und einer index.html.
- Zeit zum Ausprobieren: Fehler und Wiederholungen gehören zum Lernprozess.

Handschrift in der PDF

Die hellen Schreibflächen sind absichtlich groß und bieten Linien oder ein Code-Raster. Du kannst die PDF in eine geeignete Notiz- beziehungsweise Annotations-App importieren und dort mit dem S Pen direkt beschriften. Die PDF enthält keine zwingend erforderlichen interaktiven Formularfelder; Anmerkungen speichert deine PDF-App in einer bearbeiteten Kopie.

Zwei Lernwege, ein gemeinsames Ziel

Die allgemeinen Übungen wechseln zwischen Reiseblog, Bibliothek, Vereinsseite, Rezeptseite und anderen Beispielen. Die Praxisaufträge bringen dasselbe Wissen in dein eigenes Projekt. Dadurch lernst du nicht nur eine Vorlage nachzubauen, sondern HTML, CSS und JavaScript selbst auf neue Aufgaben zu übertragen.

Dein Lernkompass

Das Buch orientiert sich am bestehenden Entwicklungsplan, Abschnitt 1. Die fünf Schritte sind Entwicklungsziele, keine starren Zeitvorgaben.

Planschritt	Lernziel	Nachweis
01 · Werkstatt	Dateien, Editor, Git und Vorschau	Dateien bearbeiten und Version finden
02 · HTML	Semantische Seiten, Bilder, Links, Formulare	Eine unabhängige HTML-Seite erstellen
03 · CSS	Lesbares Design und responsive Layouts	Tablet und Smartphone funktionieren
04 · JavaScript	Variablen, DOM und Ereignisse	Interaktion selbst programmieren
05 · Mini-Projekt	Aufgaben hinzufügen, ändern und organisieren	Getestete erste Aufgabenverwaltung

Vorkenntnisse und Einstieg

Für den Einstieg werden ein Git-Repository, ein Editor und eine index.html eingerichtet.
Bereits vorhandene HTML-Erfahrung kann anhand der ersten Lektionen systematisch wiederholt werden.
Verständnisfragen und Übungen helfen, den tatsächlichen Kenntnisstand einzuschätzen.

WICHTIG FÜR DEN ENTWICKLUNGSPLAN

Die Browser-Speicherung per localStorage, das echte Backend, Benutzerkonten und die dauerhafte Dokumentenspeicherung gehören regulär erst zu Abschnitt 2. Für Abschnitt 1 reicht eine funktionierende Aufgabenverwaltung mit HTML, CSS und JavaScript, die du bewusst testen und erklären kannst.

Kapitelstatus

- [ ] A · Werkstatt & Grundlagen
- [ ] B · HTML: Inhalt und Struktur
- [ ] C · HTML: Formulare, Semantik & Qualität
- [ ] D · CSS: Aussehen und Responsivität
- [ ] E · JavaScript: Verhalten und Daten
- [ ] F · Mini-Projekt, Integration und Abschluss

Dein Prüfungsprotokoll

Hier sammelst du deine Ergebnisse auf einen Blick. Die Punkte allein entscheiden nicht über den Fortschritt: Halte auch fest, was du nach einer Wiederholung besser verstehst. Du kannst dieselbe Prüfung erneut bearbeiten und das Ergebnis in der zweiten Spalte vermerken.

Zwischenprüfung	Erster Versuch	Wiederholung
A · Werkstatt	___ / 20	___ / 20
B · HTML-Struktur	___ / 20	___ / 20
C · Formulare	___ / 20	___ / 20
D · CSS	___ / 20	___ / 20
E · JavaScript	___ / 20	___ / 20

Meine wichtigsten Wiederholungsthemen

NACH DEN ZWISCHENPRÜFUNGEN · S PEN

Meine praktischen Meilensteine

- [ ] Eigenständige HTML-Seite ohne Vorlage gebaut
- [ ] HTML-Formular mit passenden Labels erklärt
- [ ] CSS-Layout auf schmalem Bildschirm geprüft
- [ ] Mit JavaScript einen Eintrag selbst hinzugefügt
- [ ] Aufgabenverwaltung getestet und mit Git gesichert

Inhaltsverzeichnis

Tippe auf einen Eintrag, um direkt zur jeweiligen Einheit oder Lektion zu springen. Die Zwischenprüfungen sind im Text direkt nach den betreffenden Lernblöcken platziert.

Die Entwicklungswerkstatt 7

- Wie eine Webseite funktioniert . . . 8
- Ordner, Dateinamen und Pfade . . . 10
- Editor, Vorschau, Git und Codespaces . . . 12
- Zwischenprüfung A · Werkstatt . . . 14

HTML – Das Grundgerüst und echte Inhalte 16

- Die HTML-Dokumentstruktur . . . 17
- Tags, Attribute und Verschachtelung . . . 19
- Überschriften und sinnvolle Absätze . . . 21
- Text hervorheben und Sonderzeichen . . . 23
- Aufzählungen, Nummerierungen und Begriffe . . . 25
- Links und Navigation . . . 27
- Bilder und alternative Texte . . . 29
- Semantische Seitenbereiche . . . 31
- Tabellen für echte Tabellendaten . . . 33
- Zwischenprüfung B · HTML-Struktur . . . 35

HTML – Formulare und gute Webseiten 37

- Formulare: Das Grundprinzip . . . 38
- Eingabetypen und eingebaute Prüfung . . . 40
- Mehr Auswahl: Checkbox, Radio, Select und Textarea . . . 42
- Formulare senden: GET, POST und Buttons . . . 44
- Weitere HTML-Inhalte und Medien . . . 46
- Barrierefreiheit, Sprache und gute Struktur . . . 48
- Fehler finden und HTML prüfen . . . 50
- Zwischenprüfung C · Formulare . . . 52

CSS – Unser eigenes Designsystem 54

- CSS einbinden und Regeln verstehen . . . 55
- Farben, Schriften und Einheiten . . . 57
- Das Box-Modell: Abstand ist nicht gleich Abstand . . . 59
- Selektoren und die Kaskade . . . 61

Flexbox: Inhalte in einer Richtung anordnen . . . 63
CSS Grid und responsive Seiten . . . 65
Interaktionen schön und zugänglich gestalten . . . 67
Zwischenprüfung D · CSS . . . 69

JavaScript – Die Webseite reagiert 71

JavaScript sicher einbinden und ausprobieren . . . 72
Variablen und Datentypen . . . 74
Vergleiche und Bedingungen . . . 76
Arrays, Objekte und Schleifen . . . 78
Funktionen und Ereignisse . . . 80
Der DOM: HTML mit JavaScript verändern . . . 82
Formularereignisse und einfache Validierung . . . 84
Aufgaben bearbeiten, abhaken und löschen . . . 86
Fehlersuche und kleine Tests . . . 88
Zwischenprüfung E · JavaScript . . . 90

Das erste funktionsfähige Mini-Projekt 92

Mini-Projekt planen: Daten, Ansicht und Aktionen . . . 93
Filtern, Sortieren und Fristen verstehen . . . 95
Projektabschluss: Testen, dokumentieren, versionieren . . . 97

Ein zweites Projekt – ohne Vorlage 99

Der Abschlusstest 100

Dein persönliches Ergebnis 107

Die wichtigsten Begriffe 109

HTML-Spickzettel 110

CSS- & JavaScript-Spickzettel 111

Antworten und Hinweise 112

Prüfungsmaßstab & Lösungen 117

Gute Quellen zum Nachschlagen 118

ABSCHNITT 01 / LEHRBUCHEINHEIT A

Die Entwicklungswerkstatt

Du verstehst den Weg vom Quelltext zur Webseite, kannst deinen Editor bedienen und Git für einen nachvollziehbaren Projektstand einsetzen.

EINORDNUNG IM ENTWICKLUNGSPLAN

Entwicklungsplan: Schritt 01 von 25. In dieser Einheit lernst du Grundlagen, die ausdrücklich auch für andere Websites einsetzbar sind. Die Praxisfelder übertragen das Wissen anschließend auf deinen Prototyp.

Das wirst du in dieser Einheit erarbeiten

01 Wie eine Webseite funktioniert

02 Ordner, Dateinamen und Pfade

03 Editor, Vorschau, Git und Codespaces

Lies nicht alles nur am Stück: Nach jeder Lektion folgt eine eigene Stift- und Praxisseite.

EINHEIT A / SCHRITT 1 VON 25 / LEKTION 1

Wie eine Webseite funktioniert

Nach dieser Lektion kannst du ...

- Browser, Datei und Server unterscheiden
- Den Weg von einer URL zur Anzeige erklären

Verstehen

Eine Webseite besteht nicht aus einem einzelnen Bild. Der Browser erhält Dateien und baut daraus eine Darstellung. HTML beschreibt die Bedeutung und Struktur, CSS die Gestaltung, JavaScript mögliche Reaktionen. Viele Seiten funktionieren zunächst ganz ohne JavaScript.

Bei einer Internetadresse fragt dein Browser einen Server nach einer Ressource. Der Server liefert etwa eine HTML-Datei; diese kann weitere Dateien wie CSS, Bilder und JavaScript referenzieren. In der Entwicklung kann dieser Server lokal oder in einem Codespace laufen.

Das ist wichtig: Der Browser führt JavaScript normalerweise auf deinem Gerät aus. Ein Backend auf dem Server ist etwas anderes. Beim ersten Mini-Projekt beginnen wir mit dem Browser und ergänzen den produktiven Server erst später.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
index.html       → Inhalt und Struktur
style.css        → Aussehen
script.js        → Verhalten
Browser          → setzt alles zusammen
```

MERKE & TYPISCHE STOLPERFALLE

HTML ist eine Auszeichnungssprache; CSS eine Stylesheet-Sprache; JavaScript eine Programmiersprache. Die drei Werkzeuge ergänzen sich, sind aber nicht dasselbe.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / A · 1

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Zeichne ein einfaches Ablaufdiagramm: Nutzer → Browser → Server → HTML/CSS/JS → sichtbare Seite. Beschrifte die Rollen.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Öffne deine index.html und markiere gedanklich: Was gehört zur Struktur, was später zum Design und was zum Verhalten?

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum genügt eine reine HTML-Datei für eine statische Informationsseite?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT A / SCHRITT 1 VON 25 / LEKTION 2

Ordner, Dateinamen und Pfade

Nach dieser Lektion kannst du ...

- Projektstruktur verstehen
- Relative Pfade selbst lesen

Verstehen

Ein Projekt ist ein Verzeichnis mit zusammengehörenden Dateien. Der Browser findet Dateien anhand ihres Pfads. Liegt style.css im selben Ordner wie index.html, genügt der Dateiname; liegt ein Bild im Unterordner bilder, wird bilder/foto.jpg benötigt.

Dateinamen und Groß-/Kleinschreibung müssen stimmen. Unter Linux sind Bild.jpg und bild.jpg unterschiedliche Dateien. Vermeide Leerzeichen, Umlaute und wechselnde Schreibweisen in technischen Dateinamen.

Der Browser verwendet in URLs normalerweise Schrägstriche nach vorn /. Der Dateipfad auf deinem Entwicklungsrechner ist nicht automatisch eine öffentlich erreichbare URL.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
projekt/
  index.html
  style.css
  script.js
  bilder/
    lernplatz.jpg
```

MERKE & TYPISCHE STOLPERFALLE

index.html heißt oft Startdatei. Ein Ordnername ist keine HTML-Anweisung; ein Link zu einer Datei setzt voraus, dass die Datei wirklich existiert.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / A · 2

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe für die abgebildete Ordnerstruktur den relativen Pfad von index.html zu lernplatz.jpg auf. Ergänze einen weiteren Unterordner mit einer zweiten Seite.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Lege in deinem Projekt noch keine unnötigen Ordner an. Prüfe nur, ob README.md und index.html im Projektverzeichnis liegen.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Welchen Pfad verwendest du von index.html aus für ein Bild im Ordner bilder?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT A / SCHRITT 1 VON 25 / LEKTION 3

Editor, Vorschau, Git und Codespaces

Nach dieser Lektion kannst du ...

- Speichern, Commit und Push unterscheiden
- Eine Webseite sicher testen

Verstehen

Der Editor zeigt deine Dateien; das Terminal nimmt Befehle entgegen. Mit Strg+S speicherst du im Codespace. Ein Git-Commit dokumentiert einen Projektstand; Push überträgt lokale Commits in das entfernte GitHub-Repository.

Ein einfacher Testserver lässt sich im Projektordner mit Python starten. In Codespaces erscheint der verwendete Port unter „Ports“. Nutze für Lernprojekte private Portfreigaben; echte Passwörter, Zugangsschlüssel und persönliche Dokumente gehören nicht ins Repository.

Codespaces laufen auf einem Cloudrechner und haben Nutzungskontingente. Stoppe ungenutzte Instanzen. Das ist unabhängig davon, dass die fertige Webanwendung später auf eurem Heimserver betrieben werden soll.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
python3 -m http.server 8000

# Port 8000 in „Ports“ als privat öffnen
# Server beenden: Strg+C im Terminal
```

MERKE & TYPISCHE STOLPERFALLE

Der einfache Python-Webserver dient nur zum Lernen und Testen, nicht als öffentliches Produktiv-Backend. Ein Git-Commit ersetzt kein vollständiges Backup persönlicher Daten.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / A · 3

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe in drei Sätzen den Unterschied zwischen Speichern, Commit und Push. Notiere danach die Schritte zum Beenden eines laufenden Testservers.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Starte für index.html eine Browser-Vorschau auf Port 8000. Fotografiere oder screenshotte anschließend das sichtbare Ergebnis und prüfe, ob der Inhalt deiner Datei entspricht.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum darf man „Strg+S gedrückt“ nicht mit „Code bereits auf GitHub gesichert“ gleichsetzen?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

WISSENSPRÜFUNG / NACH EINHEIT A

Zwischenprüfung A · Werkstatt

Beantworte die fünf Fragen möglichst ohne Nachschlagen. Jede richtige, verständliche Antwort zählt 2 Punkte. Für die anschließende Schreibaufgabe gibt es 10 Punkte. Insgesamt sind 20 Punkte möglich. Die Lösungshinweise findest du erst am Ende des Buches.

1. Was ist der Unterschied zwischen HTML, CSS und JavaScript?

FRAGE 1 · 2 PUNKTE

2. Was ist ein relativer Dateipfad?

FRAGE 2 · 2 PUNKTE

3. Worin unterscheiden sich Speichern, Commit und Push?

FRAGE 3 · 2 PUNKTE

4. Wofür verwendet man den Port 8000 in der Lernumgebung?

FRAGE 4 · 2 PUNKTE

5. Warum gehören Passwörter nicht ins Repository?

FRAGE 5 · 2 PUNKTE

Wenn du bei einer Frage unsicher bist, markiere sie. Genau dort lohnt sich eine gezielte Wiederholung.

WISSENSPRÜFUNG / PRAXISTEIL

Anwenden statt auswendig lernen

Skizziere einen Projektbaum mit index.html, style.css, script.js und einem Bild.

DEINE LÖSUNG · 10 PUNKTE

Selbstbewertung

Wissensfragen: ____ / 10 Schreibaufgabe: ____ / 10 Gesamt: ____ / 20

WAS WAR SCHWIERIG? WELCHE LEKTION WIEDERHOLST DU?

EINORDNUNG

16–20 Punkte: gute Grundlage; 11–15: einzelne Lektionen gezielt wiederholen; 0–10: noch einmal in Ruhe anhand eigener Beispiele üben. Diese Schwellen sind Lernhilfen, keine amtlichen Noten.

ABSCHNITT 01 / LEHRBUCHEINHEIT B

HTML – Das Grundgerüst und echte Inhalte

Du kannst Seiten unabhängig von einem konkreten Projekt semantisch aufbauen: Texte, Listen, Links, Bilder, Navigation und Tabellen.

EINORDNUNG IM ENTWICKLUNGSPLAN

Entwicklungsplan: Schritt 02 von 25. In dieser Einheit lernst du Grundlagen, die ausdrücklich auch für andere Websites einsetzbar sind. Die Praxisfelder übertragen das Wissen anschließend auf deinen Prototyp.

Das wirst du in dieser Einheit erarbeiten

01 Die HTML-Dokumentstruktur

02 Tags, Attribute und Verschachtelung

03 Überschriften und sinnvolle Absätze

04 Text hervorheben und Sonderzeichen

05 Aufzählungen, Nummerierungen und Begriffe

06 Links und Navigation

07 Bilder und alternative Texte

08 Semantische Seitenbereiche

09 Tabellen für echte Tabellendaten

Lies nicht alles nur am Stück: Nach jeder Lektion folgt eine eigene Stift- und Praxisseite.

EINHEIT B / SCHRITT 2 VON 25 / LEKTION 1

Die HTML-Dokumentstruktur

Nach dieser Lektion kannst du ...

- Grundgerüst frei schreiben
- DOCTYPE, html, head und body zuordnen

Verstehen

HTML-Dokumente haben eine verschachtelte Struktur. Die erste Zeile <!DOCTYPE html> aktiviert den modernen HTML-Modus. Das html-Element ist das Wurzelelement, sein lang-Attribut benennt die Hauptsprache der Seite.

Im head stehen Metadaten über die Seite, unter anderem title, charset und viewport. Im body steht der sichtbare und bedienbare Seiteninhalt. Eine sichtbare Hauptüberschrift kommt deshalb in body und nicht in head.

Einige Elemente besitzen öffnende und schließende Tags. Andere, zum Beispiel meta, sind sogenannte leere Elemente (Void-Elemente) und werden nicht mit einem eigenen Endtag geschlossen.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<!DOCTYPE html>
<html lang="de">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1">
    <title>Meine Lernseite</title>
  </head>
  <body>
    <h1>Willkommen</h1>
  </body>
</html>
```

MERKE & TYPISCHE STOLPERFALLE

Einrückungen verbessern die Lesbarkeit; sie bestimmen nicht allein die Bedeutung. Ein falsch verschachtelter Tag kann die Struktur dagegen verändern.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / B · 1

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe aus dem Gedächtnis ein vollständiges Grundgerüst für eine Seite mit dem Titel „Waldtagebuch“ und einer sichtbaren Hauptüberschrift.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Prüfe in deiner index.html alle öffnenden und schließenden Tags sowie den Unterschied von title und h1.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Welche Information erscheint als Browser-Tab-Titel, welche als sichtbare Überschrift?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT B / SCHRITT 2 VON 25 / LEKTION 2

Tags, Attribute und Verschachtelung

Nach dieser Lektion kannst du ...

- Tags und Elemente unterscheiden
- Attribute korrekt notieren

Verstehen

Ein HTML-Element besteht oft aus Starttag, Inhalt und Endtag. Attribute sind zusätzliche Angaben im Starttag. Beispiel: lang="de" beschreibt eine Sprache, id="kontakt" eine eindeutige Kennung und class="karte" eine wiederverwendbare CSS-Klasse.

Verschachtelung bedeutet, dass ein Element andere Elemente enthalten kann. Öffnet man zuerst p und darin strong, muss strong vor p wieder geschlossen werden. Viele browserseitige Korrekturen verstecken Fehler; deshalb prüfen wir den Quellcode bewusst.

Attribute sollten sinnvolle Namen und korrekt geschlossene Anführungszeichen besitzen. Ein id-Wert darf innerhalb einer Seite nicht mehrfach verwendet werden.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<p>Heute lerne ich <strong>HTML</strong>.</p>

<p class="hinweis" id="start">
  Eine wichtige Nachricht.
</p>
```

MERKE & TYPISCHE STOLPERFALLE

Falsch: Wichtig . Richtig: Wichtig . Das zuletzt geöffnete verschachtelte Element wird zuerst geschlossen.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / B · 2

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Markiere bei einem selbst erfundenen HTML-Element den öffnenden Tag, den Inhalt, das Attribut und den schließenden Tag. Schreibe ein richtig und ein falsch verschachteltes Beispiel.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Füge in deinem bestehenden body einem Textabsatz eine class hinzu, zum Beispiel class="einleitung". Das verändert vorerst noch nicht sein Aussehen.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum sollte eine id nicht mehrmals für verschiedene Elemente vergeben werden?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT B / SCHRITT 2 VON 25 / LEKTION 3

Überschriften und sinnvolle Absätze

Nach dieser Lektion kannst du ...

- h1 bis h6 semantisch verwenden
- Text in Absätzen strukturieren

Verstehen

Überschriften sind nicht bloß große Schrift. Sie gliedern den Inhalt: h1 bezeichnet die Hauptüberschrift, h2 die nächste Ebene, h3 eine Untergliederung von h2. Der Browser zeigt sie in Standardgrößen an; das Aussehen können wir später mit CSS ändern.

Mit p werden zusammengehörende Sätze als Absatz ausgezeichnet. Verwende Überschriften nach Inhaltsstruktur und überspringe Ebenen nicht nur, weil dir eine bestimmte Standard-Schriftgröße besser gefällt.

Für eine verständliche Seite sollten Überschriften aussagekräftig sein. Eine logische Reihenfolge unterstützt besonders Menschen mit assistiver Technik und erleichtert die spätere Wartung.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<h1>Mein Reiseblog</h1>
<h2>Reiseziele</h2>
<h3>Berge</h3>
<p>Hier sammle ich Wanderungen.</p>
<h3>Meer</h3>
<p>Hier sammle ich Küstenorte.</p>
```

MERKE & TYPISCHE STOLPERFALLE

„h3 ist kleiner als h2“ beschreibt nur das Standarddesign. Der eigentliche Unterschied ist die Gliederungsebene.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / B · 3

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Entwirf eine Überschriftenhierarchie für eine Webseite über ein Hobby mit mindestens einer h1, zwei h2 und einer h3. Ergänze zwei kurze Absätze.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Erweitere unter „Meine Aufgaben“ bei Bedarf eine Unterebene „Heute“ oder „Später“, aber nur wenn die inhaltliche Gliederung sinnvoll ist.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Wieso ist es keine gute Idee, h3 allein wegen der Schriftgröße einzusetzen?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT B / SCHRITT 2 VON 25 / LEKTION 4

Text hervorheben und Sonderzeichen

Nach dieser Lektion kannst du ...

- strong und em von bloßem Aussehen unterscheiden
- Kommentare und Sonderzeichen nutzen

Verstehen

strong zeigt starke Wichtigkeit an, em legt Betonung auf einen Ausdruck. CSS ist für rein visuelle Hervorhebungen zuständig. Mit br erzeugst du einen ausdrücklich gewünschten Zeilenumbruch, etwa in einer Adresse; zwischen normale Absätze gehört meist ein neues p statt vieler br.

Ein HTML-Kommentar wird mit <!-- begonnen und mit --> beendet. Er hilft beim Strukturieren, ist aber nicht geheim: Nutzer können ihn im Quellcode sehen.

Einige Zeichen haben in HTML besondere Bedeutung. Schreibe etwa < für das Kleiner-als-Zeichen als Inhaltstext, & für ein kaufmännisches Und und " für doppelte Anführungszeichen in bestimmten Kontexten.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<p>Bitte <strong>heute</strong> erledigen.</p>
<p>Ich wollte <em>genau das</em> sagen.</p>
<p>Beispiel: 3 &lt; 5 und Tom &amp; Mia.</p>
<!-- Später eine Kontaktsektion ergänzen -->
```

MERKE & TYPISCHE STOLPERFALLE

Das hr-Element kennzeichnet einen thematischen Übergang. Es ist kein beliebiger Abstandhalter. Leerraum wird später mit CSS erzeugt.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / B · 4

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe einen Hinweis mit strong, einen Satz mit em und ein Beispiel, das die Zeichen < und & als Text anzeigt. Erkläre jede Wahl.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Kennzeichne in deinem Einleitungsabsatz ein wirklich wichtiges Wort mit strong. Vermeide Kommentare mit vertraulichen Informationen.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum ist ein HTML-Kommentar nicht geeignet, um ein Passwort zu verstecken?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT B / SCHRITT 2 VON 25 / LEKTION 5

Aufzählungen, Nummerierungen und Begriffe

Nach dieser Lektion kannst du ...

- ul, ol, li und dl richtig nutzen
- Verschachtelte Listen schreiben

Verstehen

ul steht für eine Liste ohne wichtige Reihenfolge. ol steht für eine nummerierte Reihenfolge, etwa Arbeitsschritte. Beide verwenden li als Eintrag. Für Begriffe mit Erklärungen gibt es dl mit dt (Begriff) und dd (Beschreibung).

Ein häufiger Fehler ist das direkte Einfügen von Text zwischen ul und li. Der eigentliche Listeninhalt gehört in li. Verschachtelte Listen werden wiederum innerhalb eines li angelegt.

Nummerierungen und Aufzählungszeichen sind kein Grund, eine Liste zu wählen: Nutze Listen, wenn Inhalte tatsächlich zusammengehören. Für optische Anordnung verwenden wir später CSS.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<h2>Backanleitung</h2>
<ol>
  <li>Zutaten bereitstellen</li>
  <li>Teig rühren</li>
  <li>Backen</li>
</ol>
<ul>
  <li>Mehl</li>
  <li>Milch</li>
</ul>
```

MERKE & TYPISCHE STOLPERFALLE

Merksatz: Reihenfolge wichtig → ol. Reihenfolge nicht wichtig → ul. Begriff plus Erklärung → dl.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / B · 5

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Erstelle eine geordnete Liste für „Pflanze umtopfen“ und eine ungeordnete Einkaufsliste. Ergänze zwei Begriffserklärungen als dl.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Prüfe, ob deine Liste geplanter Funktionen eine ul ist. Ergänze optional ein verschachteltes li für „Aufgaben organisieren“.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Wann wäre ol statt ul die bessere Wahl?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT B / SCHRITT 2 VON 25 / LEKTION 6

Links und Navigation

Nach dieser Lektion kannst du ...

- Absolute und relative URLs unterscheiden
- Sichere Links und Anker setzen

Verstehen

Ein Link entsteht mit a und dem Attribut href. Eine absolute URL enthält das Schema und die Domain. Eine relative URL bezeichnet eine Datei bezogen auf den Standort der aktuellen HTML-Datei. Interne Sprungziele werden über # und eine eindeutige id angesprochen.

Der sichtbare Linktext sollte das Ziel beschreiben. „Kontaktseite öffnen“ ist verständlicher als „Hier klicken“. Mit target="_blank" öffnet ein Link meist einen neuen Tab; rel="noopener noreferrer" wird dabei aus Sicherheits- und Datenschutzgründen häufig ergänzt.

Navigation bildet man später mit nav und echten a-Elementen. Eine Schaltfläche ist für Aktionen da, ein Link für Navigation.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<a href="kontakt.html">Kontaktseite</a>
<a href="https://example.org">Beispielseite</a>
<a href="#fragen">Zu den Fragen</a>

<h2 id="fragen">Häufige Fragen</h2>
```

MERKE & TYPISCHE STOLPERFALLE

Bei href="kontakt.html" muss eine passende Datei vorhanden sein. Sonst führt der Link ins Leere (404).

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / B · 6

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe drei Links: eine externe Quelle, eine zweite HTML-Seite und einen Sprung zu einer Überschrift mit id. Kennzeichne, welcher Link absolut ist.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Ergänze in deiner Webseite einen internen Sprunglink zur Überschrift „Geplante Funktionen“. Vergib dem h2 eine passende id.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Welchen Unterschied gibt es zwischen a href="seite.html" und a href="#abschnitt"?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT B / SCHRITT 2 VON 25 / LEKTION 7

Bilder und alternative Texte

Nach dieser Lektion kannst du ...

- img, src und alt richtig verwenden
- Bilder semantisch einordnen

Verstehen

Ein Bild wird durch img mit src eingebunden. Das alt-Attribut enthält eine Textalternative für Situationen, in denen das Bild nicht gesehen oder geladen werden kann. Beschreibe bei informativen Bildern deren relevante Information; rein dekorative Bilder können alt="" erhalten.

Breite und Höhe können angegeben werden, damit der Browser Platz reserviert. Die tatsächliche flexible Darstellung regeln wir später mit CSS. Ein Bild mit Begleitbeschreibung lässt sich sinnvoll in figure und figcaption setzen.

Nutze nur Bilder, die du verwenden darfst, und achte auf vernünftige Dateigrößen. Ein Dateipfad, der auf deinem Gerät lokal existiert, funktioniert nicht automatisch in Codespaces.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<figure>
  <img src="bilder/wald.jpg"
       alt="Sonnenlicht zwischen Tannen"
       width="480" height="320">
  <figcaption>Ein Waldweg am Morgen.</figcaption>
</figure>
```

MERKE & TYPISCHE STOLPERFALLE

Ein alt-Text wiederholt nicht blind „Bild von“. Er beschreibt die Funktion beziehungsweise Information des Bildes im Kontext.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / B · 7

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe für drei Fälle sinnvolle alt-Werte: Produktfoto, Diagramm mit Trend, rein dekorative Blüte. Begründe deine Wahl.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Lege erst dann einen Bilderordner an, wenn du tatsächlich ein eigenes Beispielbild verwenden möchtest. Ein sinnvoller alt-Text ist Pflicht beim Üben.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Was ist der Unterschied zwischen alt="" und einem fehlenden alt-Attribut?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT B / SCHRITT 2 VON 25 / LEKTION 8

Semantische Seitenbereiche

Nach dieser Lektion kannst du ...

- header, nav, main, section und footer verwenden
- Struktur und Design trennen

Verstehen

Semantische HTML-Elemente benennen die Rolle eines Bereichs. header bezeichnet einleitende Inhalte, nav wichtige Navigation, main den Hauptinhalt und footer ergänzende Informationen. article ist ein in sich sinnvoller Beitrag; section eine thematische Gliederung, oft mit Überschrift.

div und span sind neutrale Container, wenn kein passenderes semantisches Element existiert. Verwende sie nicht automatisch anstelle von main, nav oder article. Eine Seite sollte in der Regel nur einen sichtbaren main-Hauptbereich besitzen.

Semantik hilft Browsern, unterstützenden Technologien, Suchmaschinen und anderen Entwicklern. Sie erzeugt jedoch nicht automatisch ein fertiges Layout.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<header><h1>Stadtmagazin</h1></header>
<nav aria-label="Hauptnavigation">
  <a href="#artikel">Artikel</a>
</nav>
<main id="artikel">
  <article>
    <h2>Neuer Park</h2>
    <p>Heute eröffnet.</p>
  </article>
</main>
<footer><p>© Beispiel</p></footer>
```

MERKE & TYPISCHE STOLPERFALLE

nav darf mehrere Links enthalten, die eine Navigation bilden. Nicht jeder einzelne Link muss in nav stehen.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / B · 8

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Zeichne einen Seitenplan für eine Vereinswebseite und beschrifte header, nav, main, section/article und footer. Erkläre, warum du article wählst oder nicht.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Plane, wie dein Alltagsplaner später einen header, einen main-Bereich und eine Navigation verwenden könnte, ohne sofort alle Module zu programmieren.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Wann ist article geeigneter als ein gewöhnlicher div?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT B / SCHRITT 2 VON 25 / LEKTION 9

Tabellen für echte Tabellendaten

Nach dieser Lektion kannst du ...

- Tabellenstruktur semantisch aufbauen
- Barrierearme Tabellenüberschriften setzen

Verstehen

Eine Tabelle ist für Daten mit Zeilen und Spalten gedacht, nicht für das gesamte Seitenlayout. table umfasst Zeilen (tr), Kopfzellen (th) und Datenzellen (td). Mit caption erhält sie einen verständlichen Titel; thead und tbody strukturieren größere Tabellen.

Das Attribut scope="col" oder scope="row" kennzeichnet, ob eine Kopfzelle für eine Spalte oder eine Zeile steht. Tabellen sollten auf kleinen Displays so gestaltet werden, dass Inhalte nicht abgeschnitten werden.

Für eine optische Kartenanordnung sind CSS Grid oder Flexbox besser geeignet.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<table>
  <caption>Trainingszeiten</caption>
  <thead><tr>
    <th scope="col">Tag</th>
    <th scope="col">Beginn</th>
  </tr></thead>
  <tbody>
    <tr><td>Di</td><td>18:00</td></tr>
    <tr><td>Do</td><td>18:30</td></tr>
  </tbody>
</table>
```

MERKE & TYPISCHE STOLPERFALLE

Eine Tabelle braucht nicht für jede Zelle eine eigene Überschrift. Verwende th gezielt für die Zuordnung der Daten.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / B · 9

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe eine Tabelle für drei Bücher mit Spalten „Titel“, „Autor“ und „Jahr“. Ergänze caption und richtige th-Zellen.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Überlege, ob Kategorien deiner Aufgaben als Liste oder echte Datentabelle dargestellt werden sollten. Begründe die Wahl in deiner Notiz.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum sollte eine Navigation nicht als Tabelle umgesetzt werden?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

WISSENSPRÜFUNG / NACH EINHEIT B

Zwischenprüfung B · HTML-Struktur

Beantworte die fünf Fragen möglichst ohne Nachschlagen. Jede richtige, verständliche Antwort zählt 2 Punkte. Für die anschließende Schreibaufgabe gibt es 10 Punkte. Insgesamt sind 20 Punkte möglich. Die Lösungshinweise findest du erst am Ende des Buches.

1. Wofür steht <!DOCTYPE html>?

FRAGE 1 · 2 PUNKTE

2. Was ist der Unterschied zwischen title und h1?

FRAGE 2 · 2 PUNKTE

3. Wie lautet die korrekte Schließreihenfolge bei verschachtelten Tags?

FRAGE 3 · 2 PUNKTE

4. Wann benutzt du ol statt ul?

FRAGE 4 · 2 PUNKTE

5. Was unterscheiden href="kontakt.html" und href="#kontakt"?

FRAGE 5 · 2 PUNKTE

Wenn du bei einer Frage unsicher bist, markiere sie. Genau dort lohnt sich eine gezielte Wiederholung.

WISSENSPRÜFUNG / PRAXISTEIL

Anwenden statt auswendig lernen

Schreibe das Grundgerüst einer frei gewählten Webseite mit head, main, h1, h2 und einem internen Link.

DEINE LÖSUNG · 10 PUNKTE

Selbstbewertung

Wissensfragen: ____ / 10 Schreibaufgabe: ____ / 10 Gesamt: ____ / 20

WAS WAR SCHWIERIG? WELCHE LEKTION WIEDERHOLST DU?

EINORDNUNG

16–20 Punkte: gute Grundlage; 11–15: einzelne Lektionen gezielt wiederholen; 0–10: noch einmal in Ruhe anhand eigener Beispiele üben. Diese Schwellen sind Lernhilfen, keine amtlichen Noten.

ABSCHNITT 01 / LEHRBUCHEINHEIT C

HTML – Formulare und gute Webseiten

Du lernst Eingaben, Formulare, Medien, Zugänglichkeit und Fehlersuche – das Fundament sinnvoll nutzbarer HTML-Seiten.

EINORDNUNG IM ENTWICKLUNGSPLAN

Entwicklungsplan: Schritt 02 von 25. In dieser Einheit lernst du Grundlagen, die ausdrücklich auch für andere Websites einsetzbar sind. Die Praxisfelder übertragen das Wissen anschließend auf deinen Prototyp.

Das wirst du in dieser Einheit erarbeiten

01 Formulare: Das Grundprinzip

02 Eingabetypen und eingebaute Prüfung

03 Mehr Auswahl: Checkbox, Radio, Select und Textarea

04 Formulare senden: GET, POST und Buttons

05 Weitere HTML-Inhalte und Medien

06 Barrierefreiheit, Sprache und gute Struktur

07 Fehler finden und HTML prüfen

Lies nicht alles nur am Stück: Nach jeder Lektion folgt eine eigene Stift- und Praxisseite.

EINHEIT C / SCHRITT 2 VON 25 / LEKTION 1

Formulare: Das Grundprinzip

Nach dieser Lektion kannst du ...

- Formular und Eingabefelder unterscheiden
- label, id und name richtig zuordnen

Verstehen

Formulare sammeln Eingaben. Ein form gruppiert zusammengehörige Steuerelemente. Das Element input kann Text, Zahlen, Datum und andere Werte erfassen. Ein label benennt ein Eingabefeld; sein for-Wert muss zur id des Feldes passen.

id ist eine eindeutige Kennung im Dokument. name bezeichnet den Schlüssel, unter dem ein Formularfeldwert bei einer herkömmlichen Formularübermittlung übertragen wird. Für JavaScript können wir Felder später über id oder andere Selektoren erreichen.

Ein placeholder ist nur ein vorübergehender Eingabehinweis. Er ersetzt nie die sichtbare Beschriftung.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<form>
  <label for="vorname">Vorname</label>
  <input type="text" id="vorname"
         name="vorname" required>
  <button type="submit">Senden</button>
</form>
```

MERKE & TYPISCHE STOLPERFALLE

Ohne ein passendes Ziel beziehungsweise eine JavaScript-Verarbeitung speichert ein Formular keine Datenbankeinträge. HTML legt zunächst Oberfläche und grundlegendes Verhalten fest.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / C · 1

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Skizziere ein Kontaktformular mit Name und Nachricht. Schreibe für das Namensfeld label und input so, dass for und id übereinstimmen.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Baue unter „Meine Aufgaben“ einen deutlich beschrifteten Texteingabebereich. Er braucht noch kein funktionierendes Hinzufügen.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Wieso ist placeholder kein Ersatz für label?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT C / SCHRITT 2 VON 25 / LEKTION 2

Eingabetypen und eingebaute Prüfung

Nach dieser Lektion kannst du ...

- Geeignete input-Typen wählen
- required, min, max, maxlength verstehen

Verstehen

Das Attribut type bestimmt, welche Art von Eingabe vorgesehen ist. Häufig sind text, email, number, date, time, password, checkbox und radio. Auf mobilen Geräten kann dies außerdem die passende Bildschirmtastatur beeinflussen.

HTML bietet eingebaute Prüfungen wie required für Pflichtfelder oder min und max für numerische Grenzen. Das ist nützlich für unmittelbare Rückmeldung. Serverseitige Verarbeitung muss Eingaben trotzdem selbst überprüfen: Browserprüfungen lassen sich umgehen.

Auch inputmode, autocomplete und maxlength können die Bedienung verbessern. Die genaue Bedeutung hängt vom Eingabetyp ab.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<label for="mail">E-Mail</label>
<input type="email" id="mail"
       name="mail" autocomplete="email" required>
<label for="alter">Alter</label>
<input type="number" id="alter"
       name="alter" min="0" max="120">
```

MERKE & TYPISCHE STOLPERFALLE

type="number" eignet sich für echte Zahlen, nicht für Postleitzahlen oder Telefonnummern mit führenden Nullen. Dafür ist text beziehungsweise tel passender.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / C · 2

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Wähle sinnvolle input-Typen für: E-Mail, Geburtstag, Telefonnummer, Anzahl Gäste und Passwort. Begründe zwei davon.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Ergänze bei deinem Aufgaben-Eingabefeld required und ein sinnvolles maxlength. Warum möchtest du bei Aufgaben nicht unbegrenzt Zeichen zulassen?

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Wieso macht required ein Feld noch nicht automatisch sicher?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT C / SCHRITT 2 VON 25 / LEKTION 3

Mehr Auswahl: Checkbox, Radio, Select und Textarea

Nach dieser Lektion kannst du ...

- Unterschied zwischen Checkbox und Radio verstehen
- Mehrzeilige Textfelder verwenden

Verstehen

Eine Checkbox ist für unabhängige Ja/Nein-Entscheidungen geeignet. Radio-Buttons bilden eine Gruppe, in der normalerweise genau eine Option ausgewählt wird; dazu tragen sie denselben name-Wert. select bietet eine Auswahlliste mit option-Einträgen.

textarea ist für längere, mehrzeilige Eingaben gedacht. Mit fieldset und legend kannst du zusammengehörige Felder verständlich gruppieren, beispielsweise die bevorzugte Kontaktart.

Benutzbare Formulare brauchen eine sinnvolle Reihenfolge und verständliche sichtbare Beschriftungen.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<fieldset>
  <legend>Kontaktart</legend>
  <label><input type="radio"
    name="kontakt" value="mail"> E-Mail</label>
  <label><input type="radio"
    name="kontakt" value="tel"> Telefon</label>
</fieldset>
<label for="nachricht">Nachricht</label>
<textarea id="nachricht"
          name="nachricht"></textarea>
```

MERKE & TYPISCHE STOLPERFALLE

Bei radio bestimmt der gemeinsame name die Gruppe; id allein gruppiert Optionen nicht. Ein select kann eine kompakte Alternative für viele Möglichkeiten sein.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / C · 3

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Entwirf ein Formular für eine Kursanmeldung: Newsletter ja/nein, bevorzugter Termin (eine Wahl), Kursart (Auswahlliste), Bemerkung (mehrzeilig).

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Plane zwei passende Eingaben für deine spätere Aufgabenverwaltung: Priorität (Auswahlliste) und Erledigt (Checkbox). Noch muss die Verarbeitung nicht funktionieren.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Was unterscheidet eine Radio-Gruppe von mehreren unabhängigen Checkboxen?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT C / SCHRITT 2 VON 25 / LEKTION 4

Formulare senden: GET, POST und Buttons

Nach dieser Lektion kannst du ...

- method und action erklären
- submit und button unterscheiden

Verstehen

Ein klassisches form kann Daten an ein Ziel senden. action nennt die Zieladresse; method bestimmt das HTTP-Verfahren. GET hängt passende Formularwerte typischerweise an die URL an und eignet sich etwa für Suchanfragen. POST überträgt Formulardaten im Anfragerumpf und wird für viele Änderungen verwendet.

POST ist keine Verschlüsselung und garantiert keine sichere Speicherung. Vertrauliche Daten verlangen HTTPS, passende Zugriffskontrollen und ein verantwortungsvoll implementiertes Backend.

button type="submit" löst eine Formularübermittlung aus; type="button" tut ohne JavaScript zunächst nichts. Im späteren JavaScript-Projekt können wir das submit-Ereignis behandeln und ein Neuladen vermeiden.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<form action="/suche" method="get">
  <label for="q">Suchbegriff</label>
  <input id="q" name="q" type="search">
  <button type="submit">Suchen</button>
</form>
<button type="button">Nur eine Aktion</button>
```

MERKE & TYPISCHE STOLPERFALLE

Eine HTML-Seite ohne Backend kann Eingaben anzeigen, aber sie hat dadurch noch keine Nutzerverwaltung und keine dauerhafte serverseitige Datenspeicherung.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / C · 4

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Vergleiche zwei Situationen: Suchformular auf einer Website und Passwortänderung. Welches Verfahren ist jeweils naheliegend und warum?

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Prüfe, ob dein Aufgabenbereich später per Formular-Submit behandelt werden soll. Notiere, wie du ihn mit JavaScript verbinden würdest.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum ersetzt die Angabe method="post" weder HTTPS noch ein sicheres Backend?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT C / SCHRITT 2 VON 25 / LEKTION 5

Weitere HTML-Inhalte und Medien

Nach dieser Lektion kannst du ...

- details/summary sinnvoll einsetzen
- Audio und Video korrekt einordnen

Verstehen

Mit details und summary lassen sich ausklappbare Informationsbereiche erstellen. Das ist nützlich für häufige Fragen. audio und video können Medien mit controls zum Bedienen versehen; Quellen werden mit src oder source angegeben.

Vermeide Autoplay ohne guten Grund, besonders mit Ton. Biete für wichtige Audio- beziehungsweise Videoinhalte passende Textalternativen, Untertitel oder Transkripte. Das erhöht Zugänglichkeit und Nutzbarkeit.

Für Codebeispiele innerhalb einer Webseite sind pre und code sinnvoll. pre erhält darin Leerzeichen und Zeilenumbrüche. Die Elemente erklären Bedeutung, nicht Gestaltung.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<details>
  <summary>Welche Zeiten gelten?</summary>
  <p>Die Öffnungszeiten stehen im Kalender.</p>
</details>
<audio controls src="ton/beispiel.mp3">
  Audio wird hier nicht unterstützt.
</audio>
<pre><code>const x = 3;</code></pre>
```

MERKE & TYPISCHE STOLPERFALLE

Ein iframe bindet eine andere Seite ein. Nutze es bewusst, berücksichtige Einbettungsregeln und Datenschutz; für eigene Inhalte ist häufig normales HTML besser.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / C · 5

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe eine kleine FAQ mit details und summary. Skizziere, welche Alternative du für ein Video über ein Lernexperiment anbieten würdest.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Ergänze optional einen aufklappbaren Bereich „Hilfe zum Hinzufügen von Aufgaben“ in deiner Webseite.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Was ist der Vorteil eines Transkripts gegenüber einer ausschließlich hörbaren Erklärung?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT C / SCHRITT 2 VON 25 / LEKTION 6

Barrierefreiheit, Sprache und gute Struktur

Nach dieser Lektion kannst du ...

- Zugängliche Seiten entwerfen
- Links, Bilder und Formulare prüfen

Verstehen

Gute Webseiten sollen mit Tastatur, Touchscreen, Stift und unterstützender Technik bedienbar sein. Semantische Elemente, logische Überschriften, echte Labels, verständliche Links und aussagekräftige Alternativtexte bilden die Grundlage.

Die Dokumentensprache steht im lang-Attribut des html-Elements. Bei fremdsprachigen Abschnitten kann lang am jeweiligen Element helfen. Fokusindikatoren für die Tastatur dürfen beim späteren CSS-Styling nicht ersatzlos verschwinden.

Farbe allein darf eine wichtige Information nicht transportieren: „Dringend“ sollte nicht bloß durch einen roten Punkt erkennbar sein. Auch bei Fehlern erklären wir in Worten, was zu ändern ist.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<html lang="de">
<!-- Beispiel im body: -->
<p>Japanisch: <span lang="ja">こんにちは</span></p>
<label for="suche">Suche</label>
<input id="suche" type="search">
```

MERKE & TYPISCHE STOLPERFALLE

Semantische HTML-Elemente sind meist der erste Schritt; ARIA ergänzt bei komplexen Widgets und ersetzt korrektes Standard-HTML nicht.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / C · 6

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Prüfe die fiktive Seite: Linktext „hier“, Bild ohne alt, roter Punkt für Fehler, Feld ohne label. Notiere vier konkrete Verbesserungen.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Teste später deine Seite nur mit der Tastatur (Tab, Enter, Leertaste), bevor du sie für „fertig“ erklärst.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Weshalb ist ein sichtbarer Fehlertext besser als ausschließlich eine rote Umrandung?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT C / SCHRITT 2 VON 25 / LEKTION 7

Fehler finden und HTML prüfen

Nach dieser Lektion kannst du ...

- Typische HTML-Fehler erkennen
- Browser-Entwicklerwerkzeuge nutzen

Verstehen

Browser versuchen fehlerhaftes HTML oft trotzdem darzustellen. Das ist bequem, kann aber Denkfehler verbergen. Typische Ursachen sind fehlende Endtags, falsch verschachtelte Elemente, vertauschte id/for-Werte und fehlerhafte relative Dateipfade.

Öffne in der Vorschau die Entwicklertools beziehungsweise untersuche den DOM-Baum. Der DOM (Document Object Model) ist die vom Browser aufgebaute Struktur der Seite. Sie kann sich von deinem Quelltext unterscheiden, wenn der Browser Fehler repariert.

Ein HTML-Validator kann auf auffällige Strukturen hinweisen. Fehlermeldungen liest man aufmerksam; man verändert nicht wahllos Code, bis sie verschwinden.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<!-- FEHLER: nicht korrekt verschachtelt -->
<p><strong>Wichtig</p></strong>

<!-- RICHTIG -->
<p><strong>Wichtig</strong></p>
```

MERKE & TYPISCHE STOLPERFALLE

Kontrolliere bei jedem Fehler: Ist die Datei gespeichert? Stimmt der Dateipfad? Gibt es Tippfehler? Wurde die Browser-Vorschau aktualisiert?

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / C · 7

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Finde in einem selbst geschriebenen Beispiel drei Fehler (falscher Pfad, falsch geschlossener Tag, fehlendes Label) und notiere die Reparatur.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Öffne deine bisherige Vorschau im Browser und erkläre mit einem Screenshot, welches HTML-Element welcher sichtbare Text ist.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum ist „die Seite sieht trotzdem gut aus“ kein Beweis für korrektes HTML?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

WISSENSPRÜFUNG / NACH EINHEIT C

Zwischenprüfung C · Formulare

Beantworte die fünf Fragen möglichst ohne Nachschlagen. Jede richtige, verständliche Antwort zählt 2 Punkte. Für die anschließende Schreibaufgabe gibt es 10 Punkte. Insgesamt sind 20 Punkte möglich. Die Lösungshinweise findest du erst am Ende des Buches.

1. Welche Aufgabe haben for am label und id am input?

FRAGE 1 · 2 PUNKTE

2. Warum ist placeholder kein Ersatz für label?

FRAGE 2 · 2 PUNKTE

3. Was ist der Unterschied zwischen Checkbox und Radio-Gruppe?

FRAGE 3 · 2 PUNKTE

4. Wann ist type="submit" sinnvoll?

FRAGE 4 · 2 PUNKTE

5. Warum kann man Eingaben nicht allein im Browser sicher validieren?

FRAGE 5 · 2 PUNKTE

Wenn du bei einer Frage unsicher bist, markiere sie. Genau dort lohnt sich eine gezielte Wiederholung.

WISSENSPRÜFUNG / PRAXISTEIL

Anwenden statt auswendig lernen

Entwirf ein zugängliches Veranstaltungsformular mit Name, E-Mail, Besuchstag, einer Auswahl und einem Senden-Button.

DEINE LÖSUNG · 10 PUNKTE

Selbstbewertung

Wissensfragen: ____ / 10 Schreibaufgabe: ____ / 10 Gesamt: ____ / 20

WAS WAR SCHWIERIG? WELCHE LEKTION WIEDERHOLST DU?

EINORDNUNG

16–20 Punkte: gute Grundlage; 11–15: einzelne Lektionen gezielt wiederholen; 0–10: noch einmal in Ruhe anhand eigener Beispiele üben. Diese Schwellen sind Lernhilfen, keine amtlichen Noten.

ABSCHNITT 01 / LEHRBUCHEINHEIT D

CSS – Unser eigenes Designsystem

Du steuerst Schriften, Farben, Abstände und responsive Layouts und baust ein wiederverwendbares Design.

EINORDNUNG IM ENTWICKLUNGSPLAN

Entwicklungsplan: Schritt 03 von 25. In dieser Einheit lernst du Grundlagen, die ausdrücklich auch für andere Websites einsetzbar sind. Die Praxisfelder übertragen das Wissen anschließend auf deinen Prototyp.

Das wirst du in dieser Einheit erarbeiten

01 CSS einbinden und Regeln verstehen

02 Farben, Schriften und Einheiten

03 Das Box-Modell: Abstand ist nicht gleich Abstand

04 Selektoren und die Kaskade

05 Flexbox: Inhalte in einer Richtung anordnen

06 CSS Grid und responsive Seiten

07 Interaktionen schön und zugänglich gestalten

Lies nicht alles nur am Stück: Nach jeder Lektion folgt eine eigene Stift- und Praxisseite.

EINHEIT D / SCHRITT 3 VON 25 / LEKTION 1

CSS einbinden und Regeln verstehen

Nach dieser Lektion kannst du ...

- HTML und CSS sauber trennen
- Selektor, Eigenschaft und Wert benennen

Verstehen

CSS beschreibt, wie vorhandene HTML-Elemente dargestellt werden. Regeln bestehen aus einem Selektor und Eigenschaft-Wert-Paaren. Eine externe CSS-Datei bleibt vom HTML getrennt und kann dadurch später viele Seiten einheitlich gestalten.

Im head wird ein Stylesheet über link rel="stylesheet" eingebunden. Ein Elementselektor wie h1 betrifft alle h1-Elemente; .karte bezeichnet eine Klasse und #start eine id. Eine CSS-Klasse kann beliebig oft eingesetzt werden.

CSS steht für Cascading Style Sheets. Treffen mehrere Regeln auf ein Element, entscheiden unter anderem Herkunft, Spezifität und Reihenfolge über den effektiven Stil. Wir beginnen mit einfachen Regeln und vermeiden übermäßig komplizierte Selektoren.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<!-- HTML im head -->
<link rel="stylesheet" href="style.css">

/* CSS in style.css */
body { background-color: #fff9fc; }
h1 { color: #713f62; }
.karte { border: 1px solid #dfbfd1; }
```

MERKE & TYPISCHE STOLPERFALLE

CSS-Kommentare verwenden /* ... */. Das ist anders als <!-- ... --> in HTML.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / D · 1

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Markiere in body { color: #333; } den Selektor, die Eigenschaft und den Wert. Formuliere zwei Regeln für h1 und p.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Lege style.css an, binde sie im head ein und ändere Hintergrund- und Überschriftenfarbe. Mache einen Screenshot vorher/nachher.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum ist die Trennung von HTML und CSS bei einer Website mit mehreren Unterseiten sinnvoll?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT D / SCHRITT 3 VON 25 / LEKTION 2

Farben, Schriften und Einheiten

Nach dieser Lektion kannst du ...

- CSS-Werte lesen und sinnvoll wählen
- Lesbarkeit vor Dekoration stellen

Verstehen

Farben werden etwa als benannte Farbe, Hexwert oder rgb() angegeben. Mit font-family bestimmst du Schriftfamilien, mit font-size die Größe, mit font-weight die Stärke und mit line-height den Zeilenabstand.

px bezeichnet eine CSS-Pixeleinheit. rem orientiert sich an der Schriftgröße des Wurzelelements und ist für skalierbare Größen oft nützlich. Prozentwerte beziehen sich abhängig von der Eigenschaft auf unterschiedliche Bezugsgrößen.

Guter Kontrast und eine lesbare Grundschrift sind wichtiger als ungewöhnliche Effekte. Entscheide dich für wenige abgestimmte Farben und Schriftstufen.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
body {
  font-family: Arial, sans-serif;
  font-size: 1rem;
  line-height: 1.55;
  color: #493746;
  background: #fffafc;
}
h1 { font-size: 2rem; }
```

MERKE & TYPISCHE STOLPERFALLE

Einheiten wie rem erleichtern konsistente Anpassungen. Für längere Texte sollte die Zeilenlänge angenehm bleiben.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / D · 2

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Entwirf eine Farbpalette für eine fiktive Naturseite und lege Grundschrift, Überschriftengröße sowie Textfarbe fest. Begründe den Kontrast.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Definiere in style.css die Grundschrift, den Hintergrund und eine zweite Überschriftenstufe. Prüfe die Darstellung auf Tablet und Smartphone.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Weshalb sollte man eine sehr helle Schrift nicht auf einen fast weißen Hintergrund setzen?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT D / SCHRITT 3 VON 25 / LEKTION 3

Das Box-Modell: Abstand ist nicht gleich Abstand

Nach dieser Lektion kannst du ...

- content, padding, border und margin unterscheiden
- Box-Größen abschätzen

Verstehen

Jedes HTML-Element wird im Layout als Rechteck behandelt. Darin liegt zunächst der Inhalt. padding ist der Abstand zwischen Inhalt und Rahmen, border der Rahmen selbst und margin der Außenabstand zu anderen Elementen.

Mit box-sizing: border-box werden angegebene Breiten bei vielen Layouts leichter handhabbar, weil padding und border in der Breite enthalten sind. margin-top, padding-inline und andere Eigenschaften helfen beim feineren Abstimmen.

Unklare Abstände verursachen viele Layoutprobleme. Erkenne zunächst, welcher Abstand gemeint ist, bevor du wahllos Werte vergrößerst.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
*, *::before, *::after {
  box-sizing: border-box;
}
.karte {
  width: 280px;
  padding: 20px;
  border: 2px solid #cba6bb;
  margin: 12px;
}
```

MERKE & TYPISCHE STOLPERFALLE

Bei box-sizing: content-box wächst die äußere Breite um padding und border; bei border-box bleiben sie in der angegebenen Breite enthalten.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / D · 3

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Zeichne ein Rechteck mit den vier Zonen: Inhalt, Innenabstand, Rahmen, Außenabstand. Beschrifte jede Zone und erkläre ihre Wirkung.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Gestalte deine Aufgabenliste als ruhigen Kartenbereich mit padding, border und margin; ändere einen Wert nach dem anderen.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Welche Eigenschaft schafft Abstand zwischen Text und Kartenrahmen?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT D / SCHRITT 3 VON 25 / LEKTION 4

Selektoren und die Kaskade

Nach dieser Lektion kannst du ...

- Klassen wiederverwenden
- Überlagerte Regeln nachvollziehen

Verstehen

Ein Element kann mehreren Klassen angehören. CSS-Regeln greifen dann gemeinsam. Klassen sind für wiederholbare Komponenten sinnvoll, id-Selektoren eher für besondere einmalige Elemente. Pseudoklassen wie :hover oder :focus-visible beschreiben Zustände.

Die Kaskade bedeutet, dass konkurrierende Regeln zusammengeführt werden. Spezifität und die Reihenfolge können darüber entscheiden, welche Angabe sichtbar wird. Halte deine Selektoren möglichst einfach und nutze Klassen für eigene Komponenten.

Vermeide !important als reflexartige Reparatur. Besser ist zu verstehen, weshalb eine Regel überschrieben wurde.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
p { color: #555; }
.hinweis { color: #7a3261; }
.hinweis.stark { font-weight: bold; }

/* HTML */
<p class="hinweis stark">Merken!</p>
```

MERKE & TYPISCHE STOLPERFALLE

Ein Klassenselektor beginnt in CSS mit Punkt (.); im HTML-Attribut class steht dieser Punkt nicht.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / D · 4

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe drei CSS-Regeln für alle Absätze, eine Klasse „info“ und eine Klasse „warnung“. Erkläre, wie du sie in HTML verwendest.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Vergib deinen Aufgabenfeldern passende Klassen und definiere einheitliche Abstände. Nutze nicht für jede Aufgabe eine andere id.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum ist .karte meist geeigneter für zehn Karten als #karte?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT D / SCHRITT 3 VON 25 / LEKTION 5

Flexbox: Inhalte in einer Richtung anordnen

Nach dieser Lektion kannst du ...

- Haupt- und Querachse verstehen
- Flexbox bei kleinen UI-Gruppen einsetzen

Verstehen

Flexbox ordnet Kindelemente entlang einer Hauptachse an, standardmäßig nebeneinander. display: flex aktiviert das Layout. Mit gap definierst du Zwischenräume, justify-content die Verteilung auf der Hauptachse und align-items die Ausrichtung auf der Querachse.

Flexbox eignet sich besonders für Navigationen, Symbol-Text-Zeilen und Button-Gruppen. flex-wrap erlaubt den Umbruch, wenn nicht genug Breite vorhanden ist.

Du musst nicht jedes Layout mit Flexbox lösen. Für zweidimensionale Raster ist Grid häufig klarer.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
.aktionen {
  display: flex;
  gap: 12px;
  align-items: center;
  flex-wrap: wrap;
}
/* HTML: <div class="aktionen">...</div> */
```

MERKE & TYPISCHE STOLPERFALLE

justify-content: space-between verteilt Elemente entlang der Hauptachse mit Freiraum dazwischen. Wenn flex-direction wechselt, ändert sich die Hauptachse.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / D · 5

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Zeichne drei Schaltflächen in einer Reihe und notiere, wie sich gap, align-items und flex-wrap auf die Darstellung auswirken.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Ordne Eingabefeld und Hinzufügen-Schaltfläche mit Flexbox an. Teste bewusst eine schmale Bildschirmbreite.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Welche CSS-Eigenschaft sorgt dafür, dass Elemente bei Platzmangel umbrechen dürfen?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT D / SCHRITT 3 VON 25 / LEKTION 6

CSS Grid und responsive Seiten

Nach dieser Lektion kannst du ...

- Ein einfaches Kartenraster erstellen
- Media Queries praktisch einsetzen

Verstehen

CSS Grid hilft bei Layouts mit Zeilen und Spalten. grid-template-columns definiert Spalten; gap Abstände. Mit repeat und minmax können flexible Raster entstehen, die auf schmalen Bildschirmen weniger Spalten benötigen.

Eine Media Query aktiviert Regeln nur unter bestimmten Bedingungen, zum Beispiel bei kleiner Bildschirmbreite. Eine Webseite wird nicht allein durch viewport responsive; Inhalte und Layoutregeln müssen sich tatsächlich anpassen.

Prüfe die Lesbarkeit auf unterschiedlichen Geräten, nicht nur im breitesten Editorfenster. Tabellen, Eingaben und Schaltflächen müssen bedienbar bleiben.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
.karten {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 16px;
}
@media (max-width: 600px) {
  .karten { grid-template-columns: 1fr; }
}
```

MERKE & TYPISCHE STOLPERFALLE

1fr steht für einen Anteil des verfügbaren Rasters. Responsive Design bedeutet mehr als „alles kleiner machen“.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / D · 6

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Zeichne dasselbe Kartenlayout einmal mit zwei Spalten und einmal mit einer Spalte. Schreibe die passende Media Query.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Teste deine Webseite in schmaler Ansicht. Passe gegebenenfalls die Abstände, Buttonbreiten und Textgröße an.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Was bewirkt @media (max-width: 600px) in diesem Beispiel?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT D / SCHRITT 3 VON 25 / LEKTION 7

Interaktionen schön und zugänglich gestalten

Nach dieser Lektion kannst du ...

- hover, focus-visible und disabled unterscheiden
- CSS-Variablen für ein Designsystem nutzen

Verstehen

Zustände machen Oberflächen verständlich. :hover reagiert auf einen Zeiger, :focus-visible signalisiert tastaturbedienten Fokus. Eine Schaltfläche muss auch ohne Hover erkennbar sein; Tablets und Smartphones haben nicht immer einen Mauszeiger.

CSS Custom Properties (Variablen) ermöglichen einheitliche Farben und Abstände. In :root definierte Werte können mit var() wiederverwendet werden. So bleibt der Stil einer wachsenden Anwendung konsistent.

Nutze ausreichende Trefferflächen für Touch- und Stiftbedienung. Entferne sichtbare Fokusmarkierungen nicht ohne gleichwertigen Ersatz.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
:root {
  --rosa: #be729b;
  --pflaume: #693750;
}
button { background: var(--rosa); }
button:focus-visible {
  outline: 3px solid var(--pflaume);
  outline-offset: 3px;
}
```

MERKE & TYPISCHE STOLPERFALLE

Ein Klickzustand kann später durch JavaScript verändert werden, aber CSS gestaltet ihn. Eine nur mit :hover sichtbare Aktion ist auf Touchgeräten leicht unauffindbar.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / D · 7

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe ein kleines Variablensystem für drei Farben und zwei Abstände. Zeichne einen Button mit Normal- und Fokuszustand.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Gestalte die Hinzufügen-Schaltfläche mit gut lesbarem Text und sichtbarem Tastaturfokus.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum sollte ein Button nicht ausschließlich über :hover verständlich werden?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

WISSENSPRÜFUNG / NACH EINHEIT D

Zwischenprüfung D · CSS

Beantworte die fünf Fragen möglichst ohne Nachschlagen. Jede richtige, verständliche Antwort zählt 2 Punkte. Für die anschließende Schreibaufgabe gibt es 10 Punkte. Insgesamt sind 20 Punkte möglich. Die Lösungshinweise findest du erst am Ende des Buches.

1. Was ist der Unterschied zwischen margin und padding?

FRAGE 1 · 2 PUNKTE

2. Was bewirkt display: flex?

FRAGE 2 · 2 PUNKTE

3. Wozu nutzt man CSS Grid?

FRAGE 3 · 2 PUNKTE

4. Was ist eine Media Query?

FRAGE 4 · 2 PUNKTE

5. Warum ist :focus-visible wichtig?

FRAGE 5 · 2 PUNKTE

Wenn du bei einer Frage unsicher bist, markiere sie. Genau dort lohnt sich eine gezielte Wiederholung.

WISSENSPRÜFUNG / PRAXISTEIL

Anwenden statt auswendig lernen

Schreibe CSS für eine zweispaltige Kartensammlung, die bei schmalen Bildschirmen einspaltig wird.

DEINE LÖSUNG · 10 PUNKTE

Selbstbewertung

Wissensfragen: ____ / 10 Schreibaufgabe: ____ / 10 Gesamt: ____ / 20

WAS WAR SCHWIERIG? WELCHE LEKTION WIEDERHOLST DU?

EINORDNUNG

16–20 Punkte: gute Grundlage; 11–15: einzelne Lektionen gezielt wiederholen; 0–10: noch einmal in Ruhe anhand eigener Beispiele üben. Diese Schwellen sind Lernhilfen, keine amtlichen Noten.

ABSCHNITT 01 / LEHRBUCHEINHEIT E

JavaScript – Die Webseite reagiert

Du verwendest Variablen, Bedingungen, Listen, Funktionen und Ereignisse, um eine statische Seite interaktiv zu machen.

EINORDNUNG IM ENTWICKLUNGSPLAN

Entwicklungsplan: Schritt 04 von 25. In dieser Einheit lernst du Grundlagen, die ausdrücklich auch für andere Websites einsetzbar sind. Die Praxisfelder übertragen das Wissen anschließend auf deinen Prototyp.

Das wirst du in dieser Einheit erarbeiten

01 JavaScript sicher einbinden und ausprobieren

02 Variablen und Datentypen

03 Vergleiche und Bedingungen

04 Arrays, Objekte und Schleifen

05 Funktionen und Ereignisse

06 Der DOM: HTML mit JavaScript verändern

07 Formularereignisse und einfache Validierung

08 Aufgaben bearbeiten, abhaken und löschen

09 Fehlersuche und kleine Tests

Lies nicht alles nur am Stück: Nach jeder Lektion folgt eine eigene Stift- und Praxisseite.

EINHEIT E / SCHRITT 4 VON 25 / LEKTION 1

JavaScript sicher einbinden und ausprobieren

Nach dieser Lektion kannst du ...

- script und defer nutzen
- HTML, CSS und JavaScript zuordnen

Verstehen

JavaScript ergänzt einer Webseite Logik und Interaktionen. Lege dafür eine separate script.js an und binde sie mit script src und defer im head ein. defer sorgt bei einem externen klassischen Skript dafür, dass HTML zuerst verarbeitet wird und die Ausführung danach erfolgt.

Du kannst zunächst einfache Informationen in der Browserkonsole ausgeben. console.log hilft beim Lernen und Debuggen, zeigt aber keinen sichtbaren Seitentext an.

Fehler im JavaScript können Funktionen stoppen, während der restliche HTML-Inhalt weiter sichtbar bleibt. Daher testen wir oft in kleinen Schritten.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<!-- index.html, innerhalb von head -->
<script src="script.js" defer></script>

// script.js
console.log("Meine Webseite startet");
```

MERKE & TYPISCHE STOLPERFALLE

JavaScript-Kommentare beginnen mit // oder /* ... */. Sie unterscheiden sich von HTML- und CSS-Kommentaren.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / E · 1

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe auf, welche drei Dateien HTML, CSS und JavaScript enthalten. Erkläre, was die Konsole ist und wozu defer dient.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Lege script.js an, verknüpfe sie, und überprüfe in der Browserkonsole eine selbst geschriebene console.log-Ausgabe.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum bedeutet „HTML ist sichtbar“ nicht, dass der JavaScript-Code fehlerfrei läuft?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT E / SCHRITT 4 VON 25 / LEKTION 2

Variablen und Datentypen

Nach dieser Lektion kannst du ...

- const und let unterscheiden
- Strings, Zahlen und Boolean erkennen

Verstehen

Variablen geben Werten einen Namen. const wird verwendet, wenn die Bindung nicht neu zugewiesen werden soll; let, wenn eine spätere neue Zuweisung notwendig ist. Für den Anfang verzichten wir auf var.

Häufige Datentypen sind String (Text), Number (Zahl), Boolean (true/false) sowie null als bewusst leeren Wert. JavaScript unterscheidet "5" als Text und 5 als Zahl. Das wirkt sich auf Vergleiche und Rechnungen aus.

Namen sollten verständlich sein, zum Beispiel aufgabenAnzahl statt x. Im JavaScript-Code sind Groß-/Kleinschreibung und Zeichen exakt zu beachten.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
const titel = "Meine Einkaufsliste";
let anzahl = 3;
const erledigt = false;
anzahl = anzahl + 1;
console.log(titel, anzahl, erledigt);
```

MERKE & TYPISCHE STOLPERFALLE

const verhindert eine erneute Zuweisung der Variablen, macht ein darin gespeichertes Objekt aber nicht automatisch vollständig unveränderlich.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / E · 2

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Lege gedanklich Werte an für: Titel eines Buchs, Anzahl Seiten, ausgeliehen ja/nein. Schreibe Datentyp und sinnvolle Variablennamen.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Erstelle in script.js zwei Variablen für einen frei erfundenen Aufgabentitel und eine Priorität. Gib sie in der Konsole aus.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum ergibt "5" + 2 in JavaScript etwas anderes als 5 + 2?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT E / SCHRITT 4 VON 25 / LEKTION 3

Vergleiche und Bedingungen

Nach dieser Lektion kannst du ...

- if/else sinnvoll verwenden
- === und > unterscheiden

Verstehen

Mit if reagiert ein Programm abhängig von einer Bedingung. Die Bedingung wird als wahr oder falsch ausgewertet. else beschreibt den Alternativfall; else if zusätzliche Fälle.

Vergleichsoperatoren sind unter anderem === (strenge Gleichheit), !== (strenge Ungleichheit), >, >=, < und <=. Mit && müssen mehrere Bedingungen wahr sein; mit || genügt eine davon.

Vermeide bei Vergleichen versehentlich ein einzelnes =. Das steht für Zuweisung, nicht für Vergleich.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
const punkte = 8;
if (punkte >= 10) {
  console.log("Ziel erreicht");
} else {
  console.log("Weiter üben");
}
```

MERKE & TYPISCHE STOLPERFALLE

Die Zeichenfolge "10" ist nicht dasselbe wie die Zahl 10. === vermeidet unerwartete Typumwandlungen in vielen Vergleichen.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / E · 3

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Formuliere eine if/else-Bedingung: Wenn mindestens 60 Punkte erreicht wurden, ist der Test bestanden, sonst nicht. Schreibe die möglichen Ausgaben.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Ergänze in script.js eine kleine Konsolenprüfung: Ist die beispielhafte Priorität "hoch"? Noch ohne Eingabefeld.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Was ist der Unterschied zwischen = und ===?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT E / SCHRITT 4 VON 25 / LEKTION 4

Arrays, Objekte und Schleifen

Nach dieser Lektion kannst du ...

- Listen und strukturierte Einträge modellieren
- for...of anwenden

Verstehen

Ein Array speichert eine geordnete Sammlung von Werten. Ein Objekt fasst zusammengehörige Eigenschaften mit Namen zusammen. Eine Aufgabenliste besteht daher sinnvollerweise aus einem Array von Aufgabenobjekten.

Mit einer Schleife lässt sich jedes Element durchlaufen. for...of eignet sich für einfache Listen. Methoden wie push fügen am Ende eines Arrays einen neuen Wert hinzu; filter erstellt ein neues Array mit ausgewählten Einträgen.

Wir trennen Daten von ihrer sichtbaren Darstellung: Ein Aufgabenobjekt ist noch keine HTML-Karte. Die Darstellung bauen wir später aus seinen Eigenschaften.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
const aufgaben = [
  { titel: "Lesen", erledigt: false },
  { titel: "Zeichnen", erledigt: true }
];
for (const aufgabe of aufgaben) {
  console.log(aufgabe.titel);
}
```

MERKE & TYPISCHE STOLPERFALLE

Ein Index beginnt bei Arrays mit 0. Eigenschaften von Objekten werden zum Beispiel mit aufgabe.titel gelesen.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / E · 4

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Erstelle ein Array mit drei Lieblingsbüchern und ein Objekt zu einem Buch (titel, seiten, gelesen). Schreibe eine Schleife für alle Titel.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Schreibe zunächst zwei Aufgabenobjekte für unseren Prototyp. Füge später eine eigene Prioritätseigenschaft hinzu.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum eignet sich ein Array von Objekten besser als zehn getrennte Variablen aufgabe1, aufgabe2 ...?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT E / SCHRITT 4 VON 25 / LEKTION 5

Funktionen und Ereignisse

Nach dieser Lektion kannst du ...

- Wiederverwendbaren Code schreiben
- Event-Handler verstehen

Verstehen

Funktionen bündeln Schritte unter einem Namen. Parameter sind Eingabewerte; return gibt ein Ergebnis zurück. Ein klarer Funktionsname beschreibt, was sie tut. So entsteht weniger duplizierter Code.

Ein Ereignis (event) ist etwas, das geschieht: etwa ein Klick, eine Eingabe oder das Absenden eines Formulars. addEventListener verbindet eine Funktion mit einem Ereignis.

Statt alles sofort in einem großen Block zu programmieren, arbeiten wir mit kleinen Funktionen, die wir einzeln verstehen und testen können.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
function begruesse(name) {
  return "Hallo, " + name + "!";
}
console.log(begruesse("Mila"));

const knopf = document.querySelector("button");
knopf.addEventListener("click", () => {
  console.log("Angeklickt!");
});
```

MERKE & TYPISCHE STOLPERFALLE

Der zuletzt gezeigte Code braucht einen vorhandenen button im HTML. Ist querySelector ohne Treffer, lautet das Ergebnis null.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / E · 5

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe eine Funktion berechneSumme(a, b), die beide Zahlen addiert und zurückgibt. Erkläre die Begriffe Parameter und Rückgabewert.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Verbinde testweise deine Hinzufügen-Schaltfläche mit einem click-Listener, der nur eine Konsolenmeldung ausgibt.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Was passiert, wenn JavaScript einen Button auswählt, den es im HTML gar nicht gibt?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT E / SCHRITT 4 VON 25 / LEKTION 6

Der DOM: HTML mit JavaScript verändern

Nach dieser Lektion kannst du ...

- Elemente auswählen und Text setzen
- value und textContent unterscheiden

Verstehen

Der DOM ist die strukturierte Repräsentation deiner HTML-Seite im Browser. Über document.querySelector kannst du Elemente anhand eines CSS-Selektors finden. Mit textContent setzt du Textinhalt, mit value liest du einen Eingabewert.

Mit createElement erzeugt JavaScript neue HTML-Elemente, mit append fügst du sie an anderer Stelle ein. Das ist die Grundlage für eine dynamische Aufgabenliste.

Bei frei eingegebenem Text vermeiden wir innerHTML. textContent behandelt die Eingabe als Text statt möglicherweise ausführbaren HTML-Code.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
const feld = document.querySelector("#notiz");
const ausgabe = document.querySelector("#ausgabe");
ausgabe.textContent = feld.value;

const li = document.createElement("li");
li.textContent = "Neuer Eintrag";
document.querySelector("ul").append(li);
```

MERKE & TYPISCHE STOLPERFALLE

Unterscheide ganz klar: Datenhaltung (Array), Darstellung (DOM) und Ereignisbehandlung (addEventListener).

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / E · 6

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Zeichne den Weg: Texteingabe im input → .value → JavaScript-Funktion → neues li → sichtbare Liste. Beschrifte jeden Schritt.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Lass eine Schaltfläche den eingetippten Text als li in einer eigenen Liste anzeigen. Schreibe zuerst selbst und nutze die Beispiele nur als Hilfe.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum verwenden wir für Nutzereingaben lieber textContent als innerHTML?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT E / SCHRITT 4 VON 25 / LEKTION 7

Formularereignisse und einfache Validierung

Nach dieser Lektion kannst du ...

- submit behandeln
- Leere Eingaben abfangen

Verstehen

Bei Formularen ist das submit-Ereignis häufig sinnvoller als ein bloßer click auf den Button: Es kann auch durch die Eingabetaste ausgelöst werden. event.preventDefault() verhindert in unserer Lernanwendung die standardmäßige Formularübermittlung und das dadurch mögliche Neuladen.

Die Methode trim() entfernt Leerraum am Anfang und Ende eines Strings. Ein leerer oder nur aus Leerzeichen bestehender Aufgabentitel soll nicht gespeichert werden.

Gute Eingabevalidierung prüft nicht nur, ob ein Feld existiert, sondern ob sein Inhalt für die beabsichtigte Funktion sinnvoll ist.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
formular.addEventListener("submit", (event) => {
  event.preventDefault();
  const text = feld.value.trim();
  if (text === "") {
    meldung.textContent = "Bitte Text eingeben.";
    return;
  }
  // Hier würden wir die Aufgabe hinzufügen.
});
```

MERKE & TYPISCHE STOLPERFALLE

Das Beispiel setzt voraus, dass formular, feld und meldung vorher passende DOM-Elemente referenzieren. In echten Programmen darf kein Element „magisch“ existieren.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / E · 7

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe in Worten den Ablauf beim Absenden eines leeren Feldes. Was soll angezeigt werden, was darf nicht passieren?

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Erweitere das Aufgabenformular um eine leere-Eingabe-Prüfung und eine verständliche Fehlermeldung.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum kann submit besser sein als ein click-Listener auf dem Hinzufügen-Button?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT E / SCHRITT 4 VON 25 / LEKTION 8

Aufgaben bearbeiten, abhaken und löschen

Nach dieser Lektion kannst du ...

- Änderungen an Daten und Oberfläche abstimmen
- Eindeutige IDs für Aufgaben einsetzen

Verstehen

Sobald mehrere Aufgaben existieren, müssen wir beim Abhaken und Löschen wissen, welche gemeint ist. Jedes Aufgabenobjekt kann dafür eine eindeutige Kennung erhalten. Ein Button für „Erledigt“ ändert zunächst die passende Dateneigenschaft und löst dann die Neudarstellung aus.

Beim Bearbeiten wird ein vorhandener Datensatz geändert statt eine zweite Aufgabe erzeugt. Beim Löschen verschwindet er aus der Sammlung; Änderungen der UI sollen anschließend den Zustand korrekt widerspiegeln.

Die Zugänglichkeit bleibt wichtig: Ein Erledigt-Status darf nicht bloß durch Durchstreichen angezeigt werden, sondern braucht eine erkennbare Beschriftung oder ein geeignetes Formularsteuerelement.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
let aufgaben = [
  { id: 1, titel: "Lesen", erledigt: false }
];
function umschalten(id) {
  const eintrag = aufgaben.find(a => a.id === id);
  if (!eintrag) return;
  eintrag.erledigt = !eintrag.erledigt;
  zeigeAufgaben();
}
```

MERKE & TYPISCHE STOLPERFALLE

Das Beispiel verlangt eine eigene Funktion zeigeAufgaben(). Sie wird in der Projektwerkstatt geschrieben; hier erklären wir die einzelnen Ideen, nicht nur das Abschreiben.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / E · 8

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Entwirf in Pseudocode je einen Ablauf für „Aufgabe bearbeiten“, „Aufgabe löschen“ und „Erledigt umschalten“. Welche Daten ändern sich?

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Plane die drei Aktionen für deinen Prototyp. Beginne mit einer Aktion, teste sie vollständig und ergänze erst danach die nächste.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum genügt es nicht, beim Abhaken nur die Farbe eines DOM-Elements zu ändern?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT E / SCHRITT 4 VON 25 / LEKTION 9

Fehlersuche und kleine Tests

Nach dieser Lektion kannst du ...

- Konsole und Browser-Entwicklertools verwenden
- Nachvollziehbare Testfälle formulieren

Verstehen

Ein Syntaxfehler verhindert die korrekte Verarbeitung von JavaScript. Ein Logikfehler kann trotz laufendem Code falsche Ergebnisse erzeugen. Mit console.log, der Fehleranzeige und schrittweisem Testen grenzt du Ursachen ein.

Prüfe eine Funktion anhand normaler, leerer und ungewöhnlicher Eingaben. Beispiel: Aufgabe hinzufügen mit „Buch lesen“, leerem Text oder vielen Leerzeichen. Teste nach jeder Änderung erneut die bereits funktionierenden Aktionen.

Ein Testfall besteht aus Ausgangslage, Aktion und erwartetem Ergebnis. Damit kannst du später selbst nachweisen, dass eine Funktion korrekt reagiert.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
// Beispielhafte Erwartung
// Ausgangslage: Liste leer
// Aktion: Text "Lesen" absenden
// Erwartung: genau ein Eintrag "Lesen"

console.log("Aktuelle Aufgaben:", aufgaben);
```

MERKE & TYPISCHE STOLPERFALLE

Bei Fehlern zuerst den ersten aussagekräftigen Hinweis in der Konsole lesen. Eine Fehlermeldung ist ein Werkzeug zum Lernen, kein Zeichen, dass du ungeeignet bist.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / E · 9

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe vier Testfälle für das Aufgabenformular. Mindestens einer soll einen ungültigen Eingabewert untersuchen.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Teste deinen aktuellen Code im Browser und dokumentiere eine Sache, die funktioniert, und eine Sache, die du gezielt untersuchen möchtest.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Was unterscheidet einen Syntaxfehler von einem Logikfehler?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

WISSENSPRÜFUNG / NACH EINHEIT E

Zwischenprüfung E · JavaScript

Beantworte die fünf Fragen möglichst ohne Nachschlagen. Jede richtige, verständliche Antwort zählt 2 Punkte. Für die anschließende Schreibaufgabe gibt es 10 Punkte. Insgesamt sind 20 Punkte möglich. Die Lösungshinweise findest du erst am Ende des Buches.

1. Worin unterscheiden sich const und let?

FRAGE 1 · 2 PUNKTE

2. Was speichert ein Array, was ein Objekt?

FRAGE 2 · 2 PUNKTE

3. Was bewirkt addEventListener?

FRAGE 3 · 2 PUNKTE

4. Was ist der Unterschied zwischen .value und .textContent?

FRAGE 4 · 2 PUNKTE

5. Warum trim() beim Hinzufügen von Aufgaben?

FRAGE 5 · 2 PUNKTE

Wenn du bei einer Frage unsicher bist, markiere sie. Genau dort lohnt sich eine gezielte Wiederholung.

WISSENSPRÜFUNG / PRAXISTEIL

Anwenden statt auswendig lernen

Schreibe eine Funktion, die aus einem Texteingabefeld einen getrimmten Wert liest, leere Eingaben ablehnt und sonst einen li-Eintrag erzeugt.

DEINE LÖSUNG · 10 PUNKTE

Selbstbewertung

Wissensfragen: ____ / 10 Schreibaufgabe: ____ / 10 Gesamt: ____ / 20

WAS WAR SCHWIERIG? WELCHE LEKTION WIEDERHOLST DU?

EINORDNUNG

16–20 Punkte: gute Grundlage; 11–15: einzelne Lektionen gezielt wiederholen; 0–10: noch einmal in Ruhe anhand eigener Beispiele üben. Diese Schwellen sind Lernhilfen, keine amtlichen Noten.

ABSCHNITT 01 / LEHRBUCHEINHEIT F

Das erste funktionsfähige Mini-Projekt

Du verknüpfst dein Wissen zu einem testbaren Mini-Projekt und sicherst deinen ersten Entwicklungsmeilenstein.

EINORDNUNG IM ENTWICKLUNGSPLAN

Entwicklungsplan: Schritt 05 von 25. In dieser Einheit lernst du Grundlagen, die ausdrücklich auch für andere Websites einsetzbar sind. Die Praxisfelder übertragen das Wissen anschließend auf deinen Prototyp.

Das wirst du in dieser Einheit erarbeiten

01 Mini-Projekt planen: Daten, Ansicht und Aktionen

02 Filtern, Sortieren und Fristen verstehen

03 Projektabschluss: Testen, dokumentieren, versionieren

Lies nicht alles nur am Stück: Nach jeder Lektion folgt eine eigene Stift- und Praxisseite.

EINHEIT F / SCHRITT 5 VON 25 / LEKTION 1

Mini-Projekt planen: Daten, Ansicht und Aktionen

Nach dieser Lektion kannst du ...

- Funktionsumfang und Datenmodell trennen
- Aufgabenverwaltung in Teilschritte zerlegen

Verstehen

Unser erstes Mini-Projekt wird eine Aufgabenverwaltung. Sie soll Hinzufügen, Bearbeiten, Löschen und Abhaken ermöglichen. Zusätzlich sehen wir Kategorien, Fälligkeitsdaten, Priorität, Dringlichkeit sowie Filter und Sortierung vor. Umgesetzt wird eine Funktion nach der anderen.

Eine Aufgabe braucht ein klar beschriebenes Datenmodell. Ein Beispiel enthält id, titel, erledigt, prioritaet, faelligAm und kategorie. Dringlichkeit soll anhand der Frist abgeleitet oder bewusst manuell markiert werden; sie ist nicht mit Wichtigkeit gleichzusetzen.

Wir planen Aufgaben als kleine, überprüfbare Einheiten. Ein schönes Layout ist hilfreich, aber eine optisch perfekte Oberfläche ersetzt keine verlässliche Programmlogik.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
const beispiel = {
  id: 1,
  titel: "Bibliothek besuchen",
  erledigt: false,
  prioritaet: "mittel",
  kategorie: "Freizeit",
  faelligAm: "2026-11-03"
};
```

MERKE & TYPISCHE STOLPERFALLE

Beispieldaten sind zum Lernen da. Echte persönliche Daten werden später nur mit eindeutigem Status übernommen.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / F · 1

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Zeichne ein Datenmodell für eine Aufgabe. Ergänze zwei sinnvolle Eigenschaften und erkläre, welche davon freiwillig sein sollten.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Erstelle eine einfache To-do-Ansicht mit Formular, Aufgabenliste und einem kleinen Bereich für Filter.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Welche zwei Eigenschaften dürfen wir bei unserem Aufgabenmodell ausdrücklich nicht vermischen?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT F / SCHRITT 5 VON 25 / LEKTION 2

Filtern, Sortieren und Fristen verstehen

Nach dieser Lektion kannst du ...

- Filter und Sortierung unterscheiden
- Datum nicht mit Priorität verwechseln

Verstehen

Filtern entscheidet, welche Einträge angezeigt werden, etwa nur offene Aufgaben. Sortieren bestimmt die Reihenfolge dieser Einträge, beispielsweise nach Fälligkeit. Die Originaldaten sollten dabei nicht versehentlich verloren gehen.

Für einfache Fristen kann eine Vergleichsfunktion prüfen, ob ein Datum überfällig oder bald erreicht ist. Datums- und Zeitzonenlogik hat Tücken; wir starten deshalb mit klaren Annahmen und testen Randfälle wie „heute“, „morgen“ und „kein Datum“.

Priorität ist eine inhaltliche Bewertung. Dringlichkeit ergibt sich aus Zeitdruck beziehungsweise expliziten Vorgaben. Eine hoch priorisierte Aufgabe kann erst nächsten Monat fällig sein.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
const offene = aufgaben.filter(a => !a.erledigt);
const alphabetisch = [...offene].sort((a, b) =>
  a.titel.localeCompare(b.titel, "de")
);
```

MERKE & TYPISCHE STOLPERFALLE

Die Kopie [...offene] verhindert, dass die Sortierung dieses Beispiel-Arrays versehentlich direkt verändert. In einer richtigen App werden Filter- und Sortierlogik getrennt getestet.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / F · 2

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Ordne sechs erfundene Aufgaben nach Priorität und Frist ein. Markiere Fälle, in denen etwas wichtig, aber nicht dringend ist.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Implementiere einen Filter „Alle / Offen / Erledigt“ und danach eine Sortierung. Dokumentiere mit einem Screenshot, was du verändert hast, und begründe deine Entscheidungen.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Was ist der grundlegende Unterschied zwischen filter() und sort()?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

EINHEIT F / SCHRITT 5 VON 25 / LEKTION 3

Projektabschluss: Testen, dokumentieren, versionieren

Nach dieser Lektion kannst du ...

- Erfolgskriterien überprüfen
- Eine stabile Git-Version erstellen

Verstehen

Zum Abschluss stellst du sicher, dass die Aufgabenverwaltung im Browser funktioniert und du ihren Code erklären kannst. Wir testen normale Eingaben, leere Werte, mehrfache Einträge, Bearbeiten, Löschen, Abhaken und die Filterlogik.

Die README beschreibt knapp, was die Anwendung kann, wie man sie startet und welche Funktionen bewusst noch fehlen. Danach committen und pushen wir den nachvollziehbaren Zwischenstand. Das ist Meilenstein 1 des Entwicklungsplans.

Persistente Serverdaten, Login, umfassender Kalender und Heimserver-Backend kommen erst in Abschnitt 2 und folgenden Entwicklungsabschnitten. Sie gehören nicht als Voraussetzung zum ersten Mini-Projekt.

So sieht das im Code aus

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
git status
git add index.html style.css script.js README.md
git commit -m "Erste Aufgabenverwaltung"
git push
```

MERKE & TYPISCHE STOLPERFALLE

git add nimmt bewusst ausgewählte Dateien auf. Vor einem Commit immer mit git status kontrollieren, ob keine vertraulichen Daten, Passwörter oder unerwünschten Dateien enthalten sind.

Weiter auf der nächsten Seite: handschriftlich üben, selbst anwenden und dein Verständnis überprüfen.

ÜBUNGSBLATT / F · 3

Selbst denken. Selbst schreiben.

01 · Allgemeine Übung – unabhängig vom Projekt

Schreibe eine Testliste für deine erste Aufgabenverwaltung mit mindestens sechs Fällen und freien Feldern für Ergebnis, Fehler und Lösung.

CODE, SKIZZE ODER STICHPUNKTE · S PEN

02 · In deiner eigenen Webseite anwenden

Führe deine Tests durch, dokumentiere den Stand in README.md sowie Git-Status und Browser-Vorschau, bevor du die nächste Phase beginnst.

BEOBACHTUNG / WAS HAT FUNKTIONIERT?

03 · Mini-Verständnisfrage

Warum brauchen wir eine README und Testfälle, obwohl die Webseite auf deinem Gerät funktioniert?

ANTWORT IN EIGENEN WORTEN

Praxis-Check: Nach dem Code-Schreiben speichern, im Browser testen und bei Bedarf einen Screenshot zur eigenen Prüfung machen.

TRANSFER / NACH HTML & FORMULAREN

Ein zweites Projekt – ohne Vorlage

Bevor du nur deinen Alltagsplaner weiterentwickelst, baue eine vollständig unabhängige HTML-Seite. Du darfst aus drei Themen wählen: eine kleine Bibliotheksseite, ein Vereinsangebot oder eine Rezept- und Einkaufsseite. Sie darf noch völlig ungestaltet sein.

Pflichtbestandteile

- [ ] Vollständiges HTML-Grundgerüst mit lang, title, charset, viewport
- [ ] Logische h1/h2/h3-Struktur und mehrere Absätze
- [ ] Eine geordnete und eine ungeordnete Liste
- [ ] Eine Navigation mit mindestens zwei passenden Links
- [ ] Ein geeignetes Bild mit begründetem alt-Text (oder begründete Bild-Planung)
- [ ] Ein Formular mit sichtbaren Labels und mehreren Eingabearten
- [ ] Mindestens zwei semantische Bereiche wie main, section, footer

Woran du erkennst, dass du HTML wirklich verstanden hast

Du kannst die Bedeutung der verwendeten Elemente erklären, eine falsche Verschachtelung selbst reparieren und den Aufbau auf ein anderes Thema übertragen. Wichtig ist nicht, ob du jedes Attribut auswendig kennst, sondern dass du Strukturentscheidungen bewusst triffst und nachschlagen kannst.

DEIN SEITENAUFBAU / ENTWURF

ABSCHLUSSPRÜFUNG / ABSCHNITT 01

Der Abschlusstest

Zum Abschluss überprüfst du, ob du eigenständig eine kleine Webseite entwerfen, gestalten, interaktiv machen und nachvollziehbar testen kannst. Die Prüfung ist ausdrücklich nicht auf den Alltagsplaner beschränkt: Baue eine „Leseliste“, eine „Rezeptverwaltung“ oder eine „Packliste“.

Bearbeite möglichst ohne direktes Kopieren aus den Lektionen. Du darfst wie bei echter Programmierarbeit technische Dokumentation konsultieren, aber notiere, was du nachschlagen musstest. Die Auswertung bewertet Verständnis und Funktionsfähigkeit, nicht bloß Tippgeschwindigkeit.

Prüfungsteil	Aufgabe	Punkte
A · Grundlagen	Struktur, Semantik, Pfade, Formulare	20
B · HTML praktisch	Vollständige zugängliche Seite entwickeln	25
C · CSS	Lesbares, responsives Designsystem	20
D · JavaScript	Einträge hinzufügen und verändern	25
E · Projektqualität	Testfälle, README, Versionsstand	10
GESAMT	Selbstkontrolle und Reflexion	100

PRÜFUNGSREGELN

Schriftliche Aufgaben kannst du vollständig in dieser PDF mit dem S Pen bearbeiten. Die praktische Umsetzung geschieht in einem eigenen Git-Zweig oder einer gesonderten Übungsdatei, damit du deinen Alltagsplaner nicht versehentlich überschreibst.

ABSCHLUSSTEST / TEIL A · 20 PUNKTE

Grundverständnis beweisen

1. Skizziere ein valides HTML-Grundgerüst mit lang, head, title, meta und body. (5 P.)

DEINE ANTWORT

2. Erkläre den Unterschied zwischen semantischem main/section und neutralem div. (4 P.)

DEINE ANTWORT

3. Nenne mindestens drei Voraussetzungen für ein gut benutzbares Formular. (4 P.)

DEINE ANTWORT

4. Unterscheide relativen Link, internen Anker und externen Link. (4 P.)

DEINE ANTWORT

5. Erkläre, warum ein Browser eine fehlerhafte Seite manchmal trotzdem anzeigt. (3 P.)

DEINE ANTWORT

ABSCHLUSSTEST / TEIL B · 25 PUNKTE

HTML praktisch: Neue Webseite

Erstelle eine eigenständige HTML-Seite für Leseliste, Packliste oder Rezepte. Keine Vorlage 1:1 nachbauen. Benutze eigene Beschriftungen und eine klare Inhaltsstruktur.

- [ ] Grundgerüst, Sprache und Metadaten korrekt (5 P.)
- [ ] Sinnvolle Überschriften, Text und Listen (5 P.)
- [ ] Semantische Seitenbereiche und Navigation (5 P.)
- [ ] Beschriftetes Formular mit geeigneten Feldtypen (6 P.)
- [ ] Zugänglichkeit: Labels, Linktexte und sinnvolle Struktur (4 P.)

Dein Seitenplan

SKIZZE ODER HTML-STRUKTUR MIT HANDSCHRIFT

Dokumentiere deine Entscheidung

Welche Inhalte hast du in header, main und footer platziert? Warum?

BEGRÜNDUNG

ABSCHLUSSTEST / TEIL C · 20 PUNKTE

CSS: verständlich und responsiv

Gestalte die in Teil B gebaute Webseite mit einer eigenen Farbpalette. Die Farben dürfen sich vom Alltagsplaner unterscheiden. Entscheidend sind Lesbarkeit, Wiederverwendbarkeit und Anpassungsfähigkeit.

- [ ] Externe CSS-Datei eingebunden (3 P.)
- [ ] Konsistente Farben und lesbare Schrift (4 P.)
- [ ] Box-Modell und Innen-/Außenabstände bewusst benutzt (4 P.)
- [ ] Flex oder Grid für ein geeignetes Layout (4 P.)
- [ ] Media Query und Tastaturfokus getestet (5 P.)

Eine responsive CSS-Regel von Hand

Schreibe hier die wichtigsten CSS-Regeln für einen Kartenbereich, der von zwei Spalten auf eine Spalte wechselt.

CSS-CODE MIT S PEN

Testnotizen

DESKTOP / TABLET / SMARTPHONE: WAS IST ANDERS?

ABSCHLUSSTEST / TEIL D · 25 PUNKTE

JavaScript: Interaktion entwickeln

Erweitere deine zweite Webseite um eine einfache Eingabeliste. Ein neuer Eintrag darf nur hinzugefügt werden, wenn sinnvoller Text eingegeben wurde. Einträge sollen sich als erledigt markieren oder löschen lassen.

- [ ] JavaScript-Datei mit defer eingebunden (3 P.)
- [ ] Eingaben auslesen und leere Werte ablehnen (5 P.)
- [ ] Array/Objekt als Datenmodell verwenden (5 P.)
- [ ] DOM-Elemente sicher erzeugen und darstellen (5 P.)
- [ ] Eine vorhandene Liste durch Aktion verändern (5 P.)
- [ ] Fehlerfälle in der Konsole prüfen (2 P.)

Schreib- und Planungsfeld

Skizziere deine Datenstruktur und den Ablauf beim Klicken auf „Hinzufügen“. Schreibe mindestens eine echte JavaScript-Funktion von Hand.

JAVASCRIPT ODER ABLAUFPLAN

ABSCHLUSSTEST / TEIL E · 10 PUNKTE

Testen und erklären

Eine Webseite ist nicht fertig, weil der erste Klick einmal funktioniert hat. Dokumentiere deinen Testprozess und deinen Lernfortschritt.

Test	Erwartetes Ergebnis	Ergebnis / Notiz
Leere Eingabe	Keine neue Listeneintragung	
Normaler Text	Ein Eintrag erscheint	
Zwei Einträge	Beide vorhanden	
Erledigt umschalten	Status ändert sich	
Eintrag löschen	Richtiger Eintrag verschwindet	
Schmale Ansicht	Lesbar und bedienbar	

Bewertung: 10 Punkte

Testfälle sinnvoll dokumentiert: 4 P. · README mit Startanleitung: 3 P. · nachvollziehbarer Git-Commit: 3 P.

WELCHE FEHLER HAST DU SELBST GEFUNDEN UND BEHOBEN?

Abschluss-Reflexion

DAS KANN ICH JETZT SELBST / DAS WIEDERHOLE ICH NOCH

ABSCHLUSS / AUSWERTUNG

Dein persönliches Ergebnis

Trage deine Punkte ein und schätze zusätzlich ein, wie sicher du die Inhalte ohne Vorlage erklären kannst. Wiederhole gezielt die Themen, die dir noch Schwierigkeiten bereiten. Ein späterer zweiter Versuch ist ausdrücklich vorgesehen.

Bereich	Punkte	Selbsteinschätzung
A · Grundlagen	____ / 20	sicher / üben / unklar
B · HTML	____ / 25	sicher / üben / unklar
C · CSS	____ / 20	sicher / üben / unklar
D · JavaScript	____ / 25	sicher / üben / unklar
E · Tests und Git	____ / 10	sicher / üben / unklar
GESAMT	____ / 100	

INTERPRETATIONSHILFE

90–100: sehr sichere Grundlagen; 75–89: gute Grundlagen mit einzelnen Lücken; 60–74: mehrere Themen gezielt wiederholen; unter 60: größere Themenblöcke mit neuen Beispielen erneut üben. Maßgeblich ist, ob die praktische Anwendung und die Erklärung des eigenen Codes gelingen.

Meine nächsten Lernziele

MEINE DREI WICHTIGSTEN WIEDERHOLUNGSTHEMEN

Meilenstein 1

Nach einer gelungenen praktischen Prüfung hast du den ersten Entwicklungsabschnitt bewältigt. Das heißt nicht, dass man niemals mehr nachschlagen muss. Es heißt, dass du eigenständig eine einfache, sinnvolle Webseite aufbauen, gestalten, mit JavaScript erweitern und erklären kannst.

NACHSCHLAGEN / GRUNDWORTSCHATZ

Die wichtigsten Begriffe

HTML · Beschreibt Aufbau und Bedeutung von Webinhalten.

Element · HTML-Baustein mit möglichem Starttag, Inhalt und Endtag.

Attribut · Zusatzinformation eines Elements, etwa id oder href.

Semantik · Bedeutung und Rolle des verwendeten Elements.

CSS · Gestaltet vorhandene HTML-Elemente.

Selektor · Wählt die Elemente aus, für die CSS-Regeln gelten.

DOM · Vom Browser aufgebautes Modell der Dokumentstruktur.

Event · Ereignis wie Klick, Eingabe oder Formularabsendung.

Array · Geordnete Sammlung von Werten.

Objekt · Zusammenstellung benannter Eigenschaften.

Funktion · Wiederverwendbarer, benannter Programmablauf.

Commit · Gespeicherter Git-Projektstand.

Push · Übertragung lokaler Commits in das entfernte Repository.

Responsive · Anpassungsfähig an verschiedene Bildschirmgrößen.

Backend · Serverseitiger Teil einer Anwendung.

API · Festgelegte Schnittstelle zwischen Softwareteilen.

NACHSCHLAGEN / PRAKTISCHE REFERENZ

HTML-Spickzettel

Element	Wofür?
html / head / body	Dokument, Metadaten, Seiteninhalt
title / meta / link	Tabtitel, Metadaten, CSS-Datei
h1–h6 / p	Überschriften und Absätze
strong / em / br	Wichtigkeit, Betonung, Zeilenumbruch
ul / ol / li / dl	Listen und Begriffsdefinitionen
a / img / figure	Links, Bilder, Mediengruppe
header / nav / main / footer	Semantische Grundbereiche
section / article / div	Themenbereich, eigenständiger Beitrag, neutraler Container
table / tr / th / td	Tabellarische Daten
form / label / input / button	Formulare und Eingaben
textarea / select / option	Mehrzeiliges Textfeld und Auswahl
fieldset / legend	Gruppe zusammengehöriger Eingaben
details / summary	Aufklappbarer Informationsbereich

NACHSCHLAGEN / CSS & JAVASCRIPT

CSS- & JavaScript-Spickzettel

CSS

Schreibweise	Bedeutung
p { color: #333; }	Regel für alle p-Elemente
.karte { … }	Selektor für class="karte"
#start { … }	Selektor für id="start"
padding / margin	Innenabstand / Außenabstand
display: flex;	Eindimensionale Anordnung
display: grid;	Raster mit Zeilen und Spalten
@media (…) { … }	Bedingte Layoutregel
:focus-visible	Sichtbarer Tastaturfokus

JavaScript

Schreibweise	Bedeutung
const / let	Variablenbindungen
if / else	Verzweigung
[] / {}	Array / Objekt
function f() {}	Wiederverwendbare Funktion
addEventListener(…)	Ereignisbehandlung
querySelector(…)	Element auswählen
element.value	Eingabewert
element.textContent	Textinhalt
document.createElement(…)	Neues HTML-Element erzeugen

LÖSUNGSTEIL / ZUERST SELBST VERSUCHEN

Antworten und Hinweise

Dieser Teil ist bewusst ganz am Ende. Lies ihn nach deinen eigenen Versuchen. Bei offenen Aufgaben können viele verschiedene Lösungen korrekt sein; entscheidend sind Semantik, nachvollziehbare Struktur und funktionierendes Verhalten.

Mini-Fragen zu allen Lektionen

Einheit A – Die Entwicklungswerkstatt

01. Warum genügt eine reine HTML-Datei für eine statische Informationsseite?
HTML gibt Inhalt und Struktur vor. Für eine statische Informationsseite sind CSS und JavaScript nicht zwingend nötig.

02. Welchen Pfad verwendest du von index.html aus für ein Bild im Ordner bilder?
bilder/lernplatz.jpg – vorausgesetzt, die Datei liegt dort. Der Pfad ist relativ zum Ordner von index.html.

03. Warum darf man „Strg+S gedrückt“ nicht mit „Code bereits auf GitHub gesichert“ gleichsetzen?
Strg+S speichert im Codespace. Commit hält einen Git-Stand fest; erst Push überträgt ihn zum GitHub-Repository.

Einheit B – HTML – Das Grundgerüst und echte Inhalte

01. Welche Information erscheint als Browser-Tab-Titel, welche als sichtbare Überschrift?
Der title ist meist im Browser-Tab zu sehen. Die h1 steht als Hauptüberschrift sichtbar im body.

02. Warum sollte eine id nicht mehrmals für verschiedene Elemente vergeben werden?
Eine id muss eindeutig sein. Doppelte IDs machen beispielsweise Sprunglinks, Labels und JS-Auswahl unzuverlässig.

03. Wieso ist es keine gute Idee, h3 allein wegen der Schriftgröße einzusetzen?
Überschriften zeigen die inhaltliche Hierarchie. Die gewünschte Schriftgröße regelst du mit CSS, nicht durch h3.

04. Warum ist ein HTML-Kommentar nicht geeignet, um ein Passwort zu verstecken?
HTML-Kommentare sind im Seitenquelltext sichtbar. Ein darin notiertes Passwort wäre nicht geheim.

05. Wann wäre ol statt ul die bessere Wahl?
ol nutzt du bei einer bedeutsamen Reihenfolge, etwa bei Arbeitsschritten; ul bei einer ungeordneten Sammlung.

06. Welchen Unterschied gibt es zwischen a href="seite.html" und a href="#abschnitt"?
seite.html führt zu einer anderen Datei. #abschnitt springt zu einem Element mit id="abschnitt" auf derselben Seite.

07. Was ist der Unterschied zwischen alt="" und einem fehlenden alt-Attribut?
alt="" bezeichnet ein bewusst dekoratives Bild. Ohne alt fehlt die Festlegung einer Textalternative.

08. Wann ist article geeigneter als ein gewöhnlicher div?
article passt für einen eigenständigen Beitrag; div ist nur ein neutraler Container ohne eigene inhaltliche Rolle.

09. Warum sollte eine Navigation nicht als Tabelle umgesetzt werden?
Eine Navigation besteht aus Links, nicht aus tabellarischen Daten. nav und a sind semantisch geeigneter.

Einheit C – HTML – Formulare und gute Webseiten

01. Wieso ist placeholder kein Ersatz für label?
Ein placeholder ist nur ein Eingabehinweis und verschwindet beim Tippen. Ein label bleibt als Beschriftung verfügbar.

02. Wieso macht required ein Feld noch nicht automatisch sicher?
required kann umgangen werden und erkennt nicht alle falschen Eingaben. Ein Server muss Daten selbst prüfen.

03. Was unterscheidet eine Radio-Gruppe von mehreren unabhängigen Checkboxen?
Radio-Buttons mit gleichem name bilden eine Einfachauswahl; Checkboxen lassen sich unabhängig markieren.

04. Warum ersetzt die Angabe method="post" weder HTTPS noch ein sicheres Backend?
POST ist keine Verschlüsselung. HTTPS schützt die Übertragung; das Backend muss Daten sicher prüfen und verarbeiten.

05. Was ist der Vorteil eines Transkripts gegenüber einer ausschließlich hörbaren Erklärung?
Ein Transkript macht Gesprochenes lesbar und durchsuchbar und hilft Menschen, die den Ton nicht hören können.

06. Weshalb ist ein sichtbarer Fehlertext besser als ausschließlich eine rote Umrandung?
Fehlertext benennt das Problem und die Abhilfe. Eine rote Umrandung allein ist nicht für alle erkennbar.

07. Warum ist „die Seite sieht trotzdem gut aus“ kein Beweis für korrektes HTML?
Browser können falsches HTML automatisch reparieren. Die Seite kann gut aussehen, obwohl ihr Code ungültig ist.

Einheit D – CSS – Unser eigenes Designsystem

01. Warum ist die Trennung von HTML und CSS bei einer Website mit mehreren Unterseiten sinnvoll?
Ein gemeinsames Stylesheet gestaltet mehrere Seiten einheitlich; Änderungen sind an einer zentralen Stelle möglich.

02. Weshalb sollte man eine sehr helle Schrift nicht auf einen fast weißen Hintergrund setzen?
Zu geringer Kontrast macht Text schwer lesbar, besonders bei Sehbeeinträchtigung oder hellem Umgebungslicht.

03. Welche Eigenschaft schafft Abstand zwischen Text und Kartenrahmen?
padding ist der Innenabstand zwischen Inhalt und Rahmen. margin schafft Abstand außerhalb des Rahmens.

04. Warum ist .karte meist geeigneter für zehn Karten als #karte?
Eine class kann auf vielen Karten wiederverwendet werden. Eine id muss auf einer Seite eindeutig sein.

05. Welche CSS-Eigenschaft sorgt dafür, dass Elemente bei Platzmangel umbrechen dürfen?
flex-wrap: wrap; erlaubt Flex-Elementen bei Platzmangel, in die nächste Zeile zu wechseln.

06. Was bewirkt @media (max-width: 600px) in diesem Beispiel?
Bei einer Breite bis 600 CSS-Pixeln gelten die Regeln in der Media Query; das Raster wird hier einspaltig.

07. Warum sollte ein Button nicht ausschließlich über :hover verständlich werden?
Auf Touchscreens ist Hover oft nicht verfügbar. Schaltflächen müssen ohne Hover verständlich und nutzbar sein.

Einheit E – JavaScript – Die Webseite reagiert

01. Warum bedeutet „HTML ist sichtbar“ nicht, dass der JavaScript-Code fehlerfrei läuft?
Der Browser kann HTML trotz JavaScript-Fehlern anzeigen. Ein Skript kann unabhängig davon nicht funktionieren.

02. Warum ergibt "5" + 2 in JavaScript etwas anderes als 5 + 2?
"5" ist Text: "5" + 2 ergibt "52". 5 + 2 addiert zwei Zahlen und ergibt 7.

03. Was ist der Unterschied zwischen = und ===?
= weist einen Wert zu. === vergleicht zwei Werte auf strenge Gleichheit, also auch ohne Typumwandlung.

04. Warum eignet sich ein Array von Objekten besser als zehn getrennte Variablen aufgabe1, aufgabe2 ...?
Ein Array von Objekten ist erweiterbar und lässt sich einfach durchlaufen, suchen und filtern.

05. Was passiert, wenn JavaScript einen Button auswählt, den es im HTML gar nicht gibt?
Die Suche liefert null. Greift der Code danach etwa auf addEventListener zu, entsteht ein Fehler.

06. Warum verwenden wir für Nutzereingaben lieber textContent als innerHTML?
textContent setzt Eingaben als reinen Text ein. innerHTML interpretiert HTML und kann dadurch unsicher sein.

07. Warum kann submit besser sein als ein click-Listener auf dem Hinzufügen-Button?
submit wird auch per Eingabetaste ausgelöst und bildet die Formularaktion ab; click reagiert nur auf Klicks.

08. Warum genügt es nicht, beim Abhaken nur die Farbe eines DOM-Elements zu ändern?
Der tatsächliche Wert erledigt muss im Datenobjekt geändert werden. Nur die Farbe ändert nicht die Daten.

09. Was unterscheidet einen Syntaxfehler von einem Logikfehler?
Syntaxfehler verletzen Schreibregeln; Logikfehler ergeben trotz gültigem Code ein falsches Verhalten.

Einheit F – Das erste funktionsfähige Mini-Projekt

01. Welche zwei Eigenschaften dürfen wir bei unserem Aufgabenmodell ausdrücklich nicht vermischen?
Priorität bedeutet Wichtigkeit, Dringlichkeit den zeitlichen Handlungsbedarf. Sie sind nicht dasselbe.

02. Was ist der grundlegende Unterschied zwischen filter() und sort()?
filter() wählt Einträge aus; sort() ändert ihre Reihenfolge und verändert das Array standardmäßig direkt.

03. Warum brauchen wir eine README und Testfälle, obwohl die Webseite auf deinem Gerät funktioniert?
Die README erklärt das Projekt. Testfälle überprüfen erwartetes Verhalten, auch nach späteren Änderungen.

Lösungshinweise: Zwischenprüfung A · Werkstatt

1. HTML strukturiert Inhalte; CSS gestaltet sie; JavaScript verarbeitet Logik und Interaktion.

2. Ein Dateipfad bezogen auf den Speicherort der aktuellen Datei.

3. Speichern schreibt im Codespace; Commit hält Git-Stand fest; Push überträgt Commits nach GitHub.

4. Als Adresse eines lokalen Testservers, dessen Vorschau Codespaces weiterleiten kann.

5. Sie könnten ungewollt geteilt, protokolliert oder im Verlauf erhalten bleiben.

Bewertung (10 P.): 3 P. für index.html, style.css und script.js; 3 P. für eine sinnvolle Ordnerstruktur; 2 P. für das Bild im passenden Ordner; 2 P. für die nachvollziehbare, beschriftete Darstellung. Andere logische Ordner- und Bildnamen sind ebenso richtig.

Eine mögliche Lösung beziehungsweise ein sinnvoller Lösungsausschnitt für den Praxisteil:

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
projekt/
  index.html
  style.css
  script.js
  bilder/
    portrait.jpg
```

Lösungshinweise: Zwischenprüfung B · HTML-Struktur

1. Aktiviert den modernen HTML-Modus des Browsers.

2. title ist Metadatum und meist Tab-Titel; h1 ist sichtbare Hauptüberschrift.

3. Das zuletzt geöffnete Element wird zuerst geschlossen.

4. Bei einer inhaltlich bedeutsamen Reihenfolge.

5. Ersteres verlinkt eine andere Datei; zweiteres ein Element mit der passenden id auf der aktuellen Seite.

Bewertung (10 P.): 2 P. für doctype/html; 2 P. für head/body; 2 P. für main mit h1 und h2;
2 P. für einen funktionierenden internen Link; 2 P. für korrekte Verschachtelung und Syntax.
Inhalte und Seitenthema sind frei wählbar.

Eine mögliche Lösung beziehungsweise ein sinnvoller Lösungsausschnitt für den Praxisteil:

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<!DOCTYPE html>
<html lang="de">
<head><title>Verein</title></head>
<body>
<main>
  <h1>Vereinsseite</h1>
  <a href="#kurse">Zu den Kursen</a>
  <h2 id="kurse">Kurse</h2>
</main>
</body>
</html>
```

Hinweis: Das Beispiel zeigt nur den prüfungsrelevanten Ausschnitt; zusätzliche Metadaten, vollständige Feldgruppen oder die vorherige Definition der DOM-Variablen sind bei einer lauffähigen Webseite gegebenenfalls noch zu ergänzen.

Lösungshinweise: Zwischenprüfung C · Formulare

1. Der gleiche Wert verbindet eine sichtbare Beschriftung mit dem Feld.

2. Ein Platzhalter verschwindet und ist keine zuverlässige sichtbare Beschriftung.

3. Checkboxen sind unabhängig; Radiobuttons gleichen Namens bilden eine Einfachauswahl.

4. Wenn die Schaltfläche ein Formular absenden soll.

5. Browserprüfungen lassen sich umgehen; serverseitig muss erneut geprüft werden.

Bewertung (10 P.): 2 P. für Name und E-Mail mit Labels; 2 P. für das Datumsfeld;
2 P. für eine passende Auswahl; 2 P. für Senden; 2 P. für die korrekte Formularstruktur.
Eigene Feldnamen und Auswahlmöglichkeiten sind erlaubt.

Eine mögliche Lösung beziehungsweise ein sinnvoller Lösungsausschnitt für den Praxisteil:

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
<form>
  <label for="name">Name</label>
  <input id="name" name="name" required>
  <label for="mail">E-Mail</label>
  <input id="mail" name="mail" type="email">
  <label for="tag">Besuchstag</label><input id="tag" name="tag" type="date">
  <label for="wahl">Angebot</label><select id="wahl" name="wahl">
    <option value="a">Kurs A</option><option value="b">Kurs B</option></select>
  <button type="submit">Senden</button></form>
```

Hinweis: Das Beispiel zeigt nur den prüfungsrelevanten Ausschnitt; zusätzliche Metadaten, vollständige Feldgruppen oder die vorherige Definition der DOM-Variablen sind bei einer lauffähigen Webseite gegebenenfalls noch zu ergänzen.

Lösungshinweise: Zwischenprüfung D · CSS

1. margin ist Außenabstand; padding Innenabstand zwischen Inhalt und Rahmen.

2. Aktiviert ein eindimensionales Flex-Layout für Kinder eines Containers.

3. Für Layouts mit Zeilen und Spalten.

4. Eine Bedingung, unter der bestimmte CSS-Regeln angewendet werden.

5. Es macht Tastaturfokus wahrnehmbar und verbessert die Bedienbarkeit.

Bewertung (10 P.): 3 P. für das zweispaltige Grid; 3 P. für eine passende Media Query;
2 P. für die Umstellung auf eine Spalte; 2 P. für korrektes und verständliches CSS.
Auch eine gleichwertige responsive Lösung mit Flexbox ist möglich.

Eine mögliche Lösung beziehungsweise ein sinnvoller Lösungsausschnitt für den Praxisteil:

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
.karten { display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 1rem;
}
@media (max-width: 600px) {
  .karten { grid-template-columns: 1fr; }
}
```

Lösungshinweise: Zwischenprüfung E · JavaScript

1. const lässt keine Neuzuweisung der Bindung zu; let erlaubt sie.

2. Array: geordnete Sammlung; Objekt: benannte Eigenschaften.

3. Registriert eine Funktion für ein Ereignis an einem Element.

4. value liest/setzt Eingabefeldwert; textContent liest/setzt Textinhalt eines Elements.

5. Damit reine Leerzeicheneingaben nicht als gültige Aufgabe zählen.

Bewertung (10 P.): 2 P. für den gelesenen, getrimmten Eingabewert; 2 P. für die Leerprüfung;
2 P. für ein neues li; 2 P. für das Einfügen in die Liste; 2 P. für stimmige Syntax.
Bereits vorhandene DOM-Variablen dürfen vorausgesetzt werden.

Eine mögliche Lösung beziehungsweise ein sinnvoller Lösungsausschnitt für den Praxisteil:

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
formular.addEventListener("submit", (e) => {
  e.preventDefault();
  const text = feld.value.trim();
  if (!text) return;
  const li = document.createElement("li");
  li.textContent = text;
  liste.append(li);
});
```

Hinweis: Das Beispiel zeigt nur den prüfungsrelevanten Ausschnitt; zusätzliche Metadaten, vollständige Feldgruppen oder die vorherige Definition der DOM-Variablen sind bei einer lauffähigen Webseite gegebenenfalls noch zu ergänzen.

LÖSUNGSTEIL / ABSCHLUSSTEST

Prüfungsmaßstab & Lösungen

Teil A – Grundverständnis

- HTML-Grundgerüst: doctype, html lang, head mit meta/title, body – korrekt verschachtelt.
- main kennzeichnet den Hauptinhalt, section eine thematische Gliederung; div ist semantisch neutral.
- Labels, passende input-Typen, logische Bedienreihenfolge, verständliche Fehler und gut bedienbare Steuerelemente.
- Relativer Link: andere Datei bezogen auf aktuellen Ort; interner Anker: #id; extern: vollständige URL.
- Browser reparieren manche Syntaxfehler beim Parsen; Aussehen beweist keine gültige Struktur.

Teil B – HTML

Es gibt keine einzige Musterseite. Prüfe korrektes Grundgerüst, semantisch sinnvolle Überschriften und Bereiche, funktionierende Links, Beschriftungen und geeignete Eingabefelder. Ein rein dekoratives Aussehen ist in diesem Teil kein Bewertungskriterium.

Teil C – CSS

CODEBEISPIEL / NICHT ALLES AUSWENDIG LERNEN

```
.karten { display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 1rem;
}
@media (max-width: 600px) {
  .karten { grid-template-columns: 1fr; }
}
```

Eine andere responsive Lösung ist ebenso richtig, wenn sie nachvollziehbar funktioniert und Tastatur-/Touch-Bedienung ermöglicht.

Teil D – JavaScript

Eine mögliche Reihenfolge: Formulareingabe per submit abfangen → trimmen → ungültige Eingabe melden → Objekt in einem Array speichern → Liste mittels DOM sicher neu darstellen → Bearbeiten/Abhaken/Löschen anhand eindeutiger IDs. Entscheidend ist, dass Daten und sichtbare Darstellung konsistent bleiben.

WEITERLERNEN / NACH DEM ERSTEN ABSCHNITT

Gute Quellen zum Nachschlagen

Die Erklärungen und Übungen sind eigenständige didaktische Ergänzungen zum Entwicklungsplan. Für die technische Referenz helfen dir die folgenden offiziellen Dokumentationsseiten. Du musst sie nicht vorab vollständig lesen.

MDN · HTML
HTML-Elemente und Attribute nachschlagen.

MDN · HTML-Formulare
Formularaufbau und Eingabetypen vertiefen.

MDN · CSS
CSS-Eigenschaften, Selektoren und Layout.

MDN · JavaScript
Sprache und Browserprogrammierung nachschlagen.

MDN · Accessibility
Zugängliche Webseiten entwickeln.

GitHub · Codespaces
Codespaces, Vorschau und Versionsverwaltung.

W3C HTML Validator
HTML auf auffällige Strukturfehler prüfen.

Ausblick nach Meilenstein 1

In Abschnitt 2 des Entwicklungsplans folgen Browser-Speicherung, saubere Projektarchitektur, Veröffentlichung auf dem Heimserver, Backend, Datenbank, Nutzeranmeldung und Dokumentenarchiv. Dieses Lernbuch bereitet dich darauf vor, ohne diese Themen voreilig in die ersten HTML-Übungen hineinzuziehen.

DEINE LERNREGEL

Wenn du den eigenen Code erklären kannst, ihn auf ein anderes Beispiel übertragen und einen Fehler selbst finden kannst, hast du mehr gelernt als durch reines Kopieren. Genau das ist das Ziel dieses Arbeitsbuchs.

LERNARBEITSBUCH · ABSCHNITT 01 · FÜR DEINE PROJEKTQUELLEN